

0.0 Imports

```
In [95]: from IPython.display import Image
import pandas as pd
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt
import plotly.graph_objects as go
from IPython.display import display, HTML
import streamlit as st
import matplotlib.patches as mpatches
```

1.0 Business Understanding

1.1. Motivation/Problem (What is the problem? / Who is reporting the issue?)

In the US, Walmart has a system similar to Uber, where delivery drivers register to deliver orders placed through the Walmart website. These delivery drivers are not Walmart employees, but work independently, accepting delivery orders and receiving these orders from Walmart staff, then delivering them to the consumer's address. Many consumers have reported that certain items in their orders were not delivered, even though the system marked the delivery as complete. This raises some critical questions:

1. Delivery Driver Fraud: There is evidence that drivers may be reporting the delivery of items that did not actually reach the customer. They may be omitting or diverting items from the order, while still recording the delivery as complete.
2. System or Process Error: It may be that the problem lies in flaws in the recording system or delivery process, not limited to intentional fraud.
3. Consumer Fraud: In some cases, consumers may claim they did not receive a product that was delivered in order to request a refund.

Walmart requested the data analyst to identify the potential causes of delivery-related fraud and propose a solution to prevent it.

1.2. Problem Root Cause (Why is a solution requested?)

Customer satisfaction is decreasing due to missing items in completed deliveries.

Revenue losses are increasing due to customer refunds related to reported missing items.

1.3. Solution (How to solve the problem?):

Proceed with a descriptive analysis to identify risk patterns and anomalous behaviors, mainly concerning customers, drivers, and potential system errors, that may indicate fraudulent delivery activities.

Develop a fraud risk score framework to estimate the likelihood of risk in future deliveries.

1.4. Deliverable:

Presentation file including visualization charts and insights about the main causes of missing items in deliveries.

Interactive dashboard for delivery monitoring.

Fraud risk score framework to identify high-risk delivery patterns.

1.5. Tools:

Python: Exploratory Data Analysis and risk metric development.

Power BI: Interactive data visualization.

PowerPoint: Executive presentation of insights.

2.0 Data Understanding

The dataset is composed of five distinct tables, each operating at a different level of granularity. Understanding these differences is essential to avoid incorrect aggregations and misleading conclusions during the analysis.

2.1 Orders Table

The orders table represents delivery orders placed through Walmart's e-commerce platform. Each row corresponds to a single order and contains information such as order value, delivery region, delivery time, and the number of items delivered and reported as missing.

It is important to note that the presence of missing items (`items_missing > 0`) does not confirm fraud, but rather indicates a reported delivery discrepancy.

2.2 Missing Items Data

The `missing_items_data` table contains information only about items reported by customers as not received. This table does not represent the full composition of an order, but only the subset of products that were declared missing.

As a result, this table should not be interpreted as a complete list of items per order, but rather as a complaint-level dataset.

2.3 Drivers Data

The `drivers_data` table provides demographic and operational information about delivery drivers, including age and total number of trips performed during the year. Each driver may be associated with multiple delivery orders.

2.4 Customers Data

The `customers_data` table contains demographic information about customers who placed orders. Similar to drivers, each customer may be associated with multiple orders over time.

2.5 Products Data

The `products_data` table includes information about individual products, such as category and price. Product-level analysis is only possible when this table is joined with the missing items data.

2.6 Absence of Fraud Labels

The dataset does not contain an explicit target variable indicating confirmed fraud. Consequently, the analysis cannot rely on supervised machine learning techniques and instead focuses on exploratory analysis, pattern detection, and risk assessment.

3.0 Solution Strategy

The solution strategy is divided into two parts:

1. Exploratory Data Analysis - The objective is not to classify orders as fraudulent, but to prioritize investigations and preventive actions. Given the absence of confirmed fraud labels, the proposed approach focuses on identifying risk patterns and anomalous behaviors that may indicate higher likelihood of delivery-related issues.
2. Fraud risk score framework - The exploratory findings will serve as the foundation for the development of a Fraud Risk Score framework. This framework will aggregate relevant risk indicators identified during the analysis and assign a relative risk level to future delivery orders, supporting preventive monitoring and investigation prioritization.

3.1 Exploratory Data Analysis Methodology

The Fact-Dimension method is used to develop the data descriptive analysis.

3.1.1 Main (Open) Question

How can delivery-related fraud risk be identified and reduced?

Are there observable patterns in delivery data that indicate higher risk of missing items?

3.1.2 Closed Questions

Closed questions are structured, measurable questions that can be answered objectively using the available data. They guide the analytical process and support evidence-based conclusions.

They are divided into two categories:

3.1.2.1 Impact-Level Questions

These questions quantify the magnitude and financial exposure of delivery discrepancies. They measure overall volume and economic impact, but do not identify behavioral concentration or disproportionate risk patterns by themselves.

1. How many items were not delivered?

2. What is the total revenue lost due to these undelivered items?

3.1.2.2 Risk-Oriented Questions

These questions analyze proportional metrics across relevant dimensions in order to identify concentration effects, deviations from baseline behavior, and potential anomalous patterns that may indicate elevated delivery-related risk.

3. Do certain drivers present disproportionately higher missing item rates compared to the overall average?

4. Are missing item reports concentrated among a small subset of customers?

5. Are specific product categories more frequently reported as missing?

6. Do certain regions exhibit higher missing item rates?

7. Is there a relationship between delivery time (hour) and missing item occurrences?

3.1.3 Defining the fact table

order_id

items_missing

order_amount

items_delivered

revenue_lost (calculated)

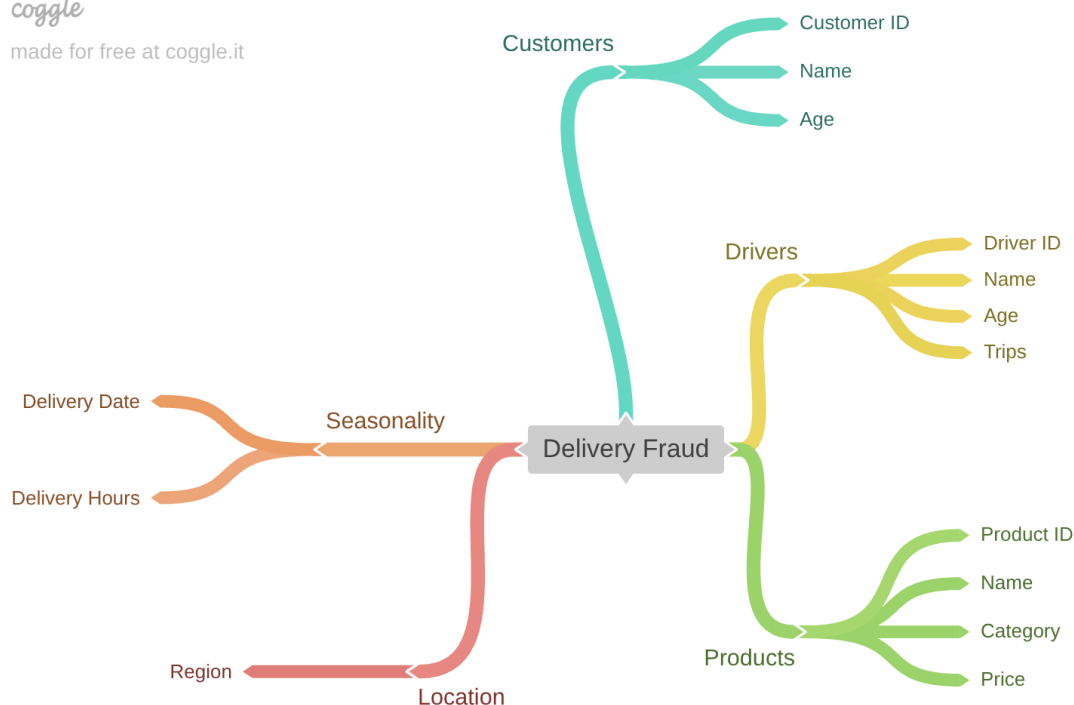
3.1.4 Defining Dimensions

In [96]: `Image('C:/Users/Igor/Repos/Walmart-Delivery-Fraud-Detection/images/delivery_frau`

Out[96]:

coggle

made for free at coggle.it



3.2 Fraud Risk Score Framework Methodology

The Fraud Risk Score framework will be developed after identifying statistically and operationally relevant risk indicators during the exploratory analysis.

The framework will:

- Select key risk variables derived from the EDA
- Normalize and standardize relevant metrics
- Combine weighted indicators into a composite risk score
- Assign relative risk levels (e.g., low, medium, high) to delivery orders

The score will not represent confirmed fraud probability, but rather a relative risk assessment tool to support preventive decision-making.

3.2.1 Objective

Identify operational and financial risk concentration across key delivery dimensions.

3.2.2 Methodology

A composite risk score was calculated combining operational and financial indicators:

Component - Meaning - Weight

incidence_rate - frequency of incidents - 15%

weighted_missing - operational severity - 15%

Average revenue loss - financial intensity - 35%

weighted_revenue_loss - financial efficiency loss - 25%

order_volume - reliability of sample - 10%

3.2.3 Risk Score Formula

Weighted normalized indicators were combined into a composite score ranging from 0 to 100, where higher values represent higher operational or financial risk.

3.2.4 Risk Classification

Score Risk Level

0–25 Low Risk

25–50 Moderate Risk

50–75 High Risk

75–100 Critical Risk

Helper Functions.

```
In [97]: # =====
# Data Description
# =====

def data_dimensions(df, dataset_name=None):
    if dataset_name:
        print(f'\n=== Data Dimensions Summary: {dataset_name} ===')
    numb_rows = df.shape[0]
    numb_cols = df.shape[1]
    print(f'Number of Rows: {numb_rows}')
    print(f'Number of Columns: {numb_cols}')

    return numb_rows, numb_cols

#--/--

def missing_values_summary(df, dataset_name=None):
    if dataset_name:
        print(f'\n=== Missing Values Summary: {dataset_name} ===')
    missing = (df.isna().sum().reset_index(name='missing_values'))
    missing['missing_%'] = missing['missing_values'] / len(df) * 100
    return missing.sort_values('missing_%', ascending=False)
```

```

#--/--

def check_unusual_values(df,
                        numeric_cols=None,
                        categorical_cols=None,
                        id_cols=None,
                        sample=11):

    print('='*70)
    print('Checking Unusual / Incoherent Values')
    print('='*70)

    # =====
    # Numerical Checks
    # =====

    if numeric_cols:
        print('\n Numerical Columns:\n')

        for col in numeric_cols:

            if df[col].dtype.kind in 'biufc':
                sorted_values = np.sort(df[col].dropna().values)

                # Convert numpy types to Python native types
                sorted_values = [v.item() if hasattr(v, 'item') else v
                                for v in sorted_values]

            if len(sorted_values) > sample * 2:
                preview = (
                    sorted_values[:sample] +
                    ['...'] +
                    sorted_values[-sample:]
                )
            else:
                preview = sorted_values

            negative_count = (df[col] < 0).sum()

            print(f'Column: {col}')
            print(f'Negative / Non-Numeroic values count: {negative_count}')
            print(f'Sample values: {preview}\n')

    # =====
    # ID duplicate checks
    # =====

    if id_cols:
        print('\n Duplicate Checks:\n')

        for col in id_cols:
            dup_count = df[col].duplicated().sum()
            dup_len = df[col].nunique() == len(df)
            print(f'Column: {col} → Duplicated values: {dup_count} → Unique valu

    # =====

```

```

# Categorical Checks
# =====

if categorical_cols:
    print('\n Categorical Columns:\n')

    for col in categorical_cols:

        unique_vals = df[col].dropna().unique()
        preview = sorted(unique_vals[:sample])

        # Convert numpy/object types
        unique_vals = [v.item() if hasattr(v, 'item') else v
                        for v in unique_vals]

        if len(unique_vals) > sample:
            preview = unique_vals[:sample] + ['...']
        else:
            preview = unique_vals

        print(f'Column: {col}')
        print(f'Unique values sample: {preview}\n')

    return

#--/--

# =====
# Descriptive Statistics
# =====

def variables_summary(df, dataset_name=None):

    print('='*70)
    if dataset_name:
        print(f'Dataset - {dataset_name}')
    print('='*70)

    # =====
    # Numerical Variables
    # =====

    num = df.select_dtypes(include=['int64', 'float64'])

    if not num.empty:

        # IQR for outliers
        Q1 = num.quantile(0.25)
        Q3 = num.quantile(0.75)
        IQR = Q3 - Q1

        outliers = ((num < (Q1 - 1.5 * IQR)) |
                    (num > (Q3 + 1.5 * IQR))).sum()

        num_summary = num.agg(['min', 'max', 'mean', 'median', 'std', 'skew', 'kurt'])
        num_summary['range'] = num_summary['max'] - num_summary['min']
        num_summary['outliers_count'] = outliers
        num_summary = num_summary.reset_index().rename(columns={'index': 'attribu

# Reorder columns

```



```

num_summary = num_summary[
    ['attribute', 'min', 'max', 'range',
     'mean', 'median', 'std', 'skew', 'kurt',
     'outliers_count']]

print('\n Numerical Summary:\n')
display(num_summary)

# =====
# Categorical Variables
# =====

cat = df.select_dtypes(exclude=['int64', 'float64', 'datetime64[ns]'])

if not cat.empty:

    cat_summary = pd.DataFrame({
        'attribute': cat.columns,
        'unique_values': cat.nunique().values,
        'most_frequent': cat.mode().iloc[0].values,
        'frequency': [
            cat[col].value_counts().iloc[0]
            for col in cat.columns
        ]
    })

    print('\n Categorical Summary:\n')
    display(cat_summary)

    return

# --/--

def variables_summary_plots(df):
    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns

    for col in numeric_cols:
        fig, axes = plt.subplots(1, 2, figsize=(12, 4))

        # Histogram
        sns.histplot(df[col], kde=True, ax=axes[0])
        axes[0].set_title(f'Histogram - {col}')

        # Boxplot
        sns.boxplot(x=df[col], ax=axes[1])
        axes[1].set_title(f'Boxplot - {col}')

        plt.tight_layout()
        plt.show()

    return

#--/--

def bar_value(plot, decimals=3, vertical=True, offset=0.01):

    for p in plot.patches:
        # Height or width of the bar
        if vertical:
            val = p.get_height()

```

```

        x = p.get_x() + p.get_width() / 2
        y = val + abs(val) * offset if pd.notnull(val) else 0
    else:
        val = p.get_width()
        x = val + abs(val) * offset if pd.notnull(val) else 0
        y = p.get_y() + p.get_height() / 2

    # Only if the value is not NaN
    if pd.notnull(val):
        plot.text(
            x=x,
            y=y,
            s=f'{val:.{decimals}f}',
            ha='center' if vertical else 'left',
            va='bottom' if vertical else 'center'
        )

# --/--

# =====
# Exploratory Data Analysis
# =====

def analyze_incidence_missing_rate(df, group_col, overall_rate, overall_avg_reve
    show_chart=True, maxBars=20, min_orders=10):

    analysis = df.groupby(group_col).agg(
        incidence_missing_rate=('missing_flag', 'mean'),
        avg_revenue_loss=('revenue_loss', 'mean'),
        total_orders=('order_id', 'count')
    ).reset_index()

    # Removing groups with low volume
    analysis = analysis[analysis['total_orders'] >= min_orders]

    analysis['above_overall_incidence_missing_rate'] = analysis['incidence_missi
    analysis['above_overall_avg_revenue_loss'] = analysis['avg_revenue_loss'] >

    total_above = analysis['above_overall_incidence_missing_rate'].sum()
    unique_items = analysis[group_col].nunique()
    pct_above = total_above / unique_items if unique_items > 0 else 0

    print(f"\n--- Analysis for: {group_col} ---")
    print(f"Total groups (after volume filter): {unique_items}")
    print(f"Groups above average: {total_above} ({pct_above:.2%})")

    if show_chart and unique_items <= maxBars:
        plt.figure(figsize=(10, 6))
        palette = {True: 'salmon', False: 'lightgrey'}

        ax = sns.barplot(
            data=analysis.sort_values('incidence_missing_rate', ascending=False)
            x=group_col,
            y='incidence_missing_rate',
            hue='above_overall_incidence_missing_rate',
            palette=palette,
            dodge=False
        )

        bar_value(ax, decimals=3)

```

```

plt.axhline(overall_rate, color='red', linestyle='--')
plt.title(f'Incidence Missing Rate by {group_col}')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

return analysis.sort_values('incidence_missing_rate', ascending=False)

# --/--

def hypothesis_metrics(df, group_col,
                      include_missing=True,
                      include_revenue=True):

    # Minimum base always present
    summary = df.groupby(group_col).agg(
        total_orders=('order_id', 'count')
    ).reset_index()

    grouped = df.groupby(group_col)

    # -----
    # Missing metrics
    # -----
    if include_missing:
        missing_metrics = grouped.agg(
            incidence_rate=('missing_flag', 'mean'),
            avg_missing=('items_missing', 'mean'),
            total_missing=('items_missing', 'sum'),
            std_missing=('items_missing', 'std'),
            total_delivered=('items_delivered', 'sum')
        ).reset_index()

        missing_metrics['miss_item_weighted_rate_%'] = (
            missing_metrics['total_missing'] /
            missing_metrics['total_delivered']
        ) * 100

        summary = summary.merge(missing_metrics, on=group_col)

    # -----
    # Revenue metrics
    # -----
    if include_revenue:
        revenue_metrics = grouped.agg(
            revenue_loss_mean=('revenue_loss', 'mean'),
            revenue_loss_total=('revenue_loss', 'sum'),
            total_order_amount=('order_amount', 'sum')
        ).reset_index()

        revenue_metrics['rev_loss_weighted_rate_%'] = (
            revenue_metrics['revenue_loss_total'] /
            revenue_metrics['total_order_amount']
        ) * 100

        summary = summary.merge(revenue_metrics, on=group_col)

    return summary

```

```

# --/--

def style_hypothesis_table(df):

    format_dict = {}
    gradient_cols = []

    if 'incidence_rate' in df.columns:
        format_dict['incidence_rate'] = '{:.4f}'
        gradient_cols.append('incidence_rate')

    if 'avg_missing' in df.columns:
        format_dict['avg_missing'] = '{:.4f}'
        gradient_cols.append('avg_missing')

    if 'miss_item_weighted_rate_%' in df.columns:
        format_dict['miss_item_weighted_rate_%'] = '{:.4f}%'
        gradient_cols.append('miss_item_weighted_rate_%')

    if 'missing_share_%' in df.columns:
        format_dict['missing_share_%'] = '{:,.4f}'
        gradient_cols.append('missing_share_%')

    if 'missing_per_1000_orders' in df.columns:
        format_dict['missing_per_1000_orders'] = '{:,.4f}'
        gradient_cols.append('missing_per_1000_orders')

    if 'revenue_loss_total' in df.columns:
        format_dict['revenue_loss_total'] = '{:,.4f}'
        gradient_cols.append('revenue_loss_total')

    if 'revenue_loss_mean' in df.columns:
        format_dict['revenue_loss_mean'] = '{:,.4f}'
        gradient_cols.append('revenue_loss_mean')

    if 'rev_loss_weighted_rate_%' in df.columns:
        format_dict['rev_loss_weighted_rate_%'] = '{:,.4f}'
        gradient_cols.append('rev_loss_weighted_rate_%')

    if 'rev_loss_per_1000_orders' in df.columns:
        format_dict['rev_loss_per_1000_orders'] = '{:,.4f}'
        gradient_cols.append('rev_loss_per_1000_orders')

    styled = df.style.format(format_dict)

    if gradient_cols:
        styled = styled.background_gradient(
            subset=gradient_cols,
            cmap='Reds'
        )

    return styled

# --/--

# displays dataset one beside other.
def display_side_by_side(dfs, captions):

    output = ""
    combined = zip(dfs, captions)

```

```

for df, cap in combined:
    # Build the HTML structure: one container for each DF + Title
    output += f"""
    <div style="margin-right: 30px; text-align: center;">
        <b style="font-size: 16px; display: block; margin-bottom: 10px;">{cap}
        {df._repr_html_()}
    </div>
    """

    # Display everything inside a flexbox (horizontal alignment)
    display(HTML(f'<div style="display: flex; align-items: flex-start;">{output}

#--/--

# =====
# Fraud Risk Score Framework
# =====

def fraud_risk_framework(df, group_col, long_format=False):

    summary = hypothesis_metrics(
        df,
        group_col,
        include_missing=True,
        include_revenue=True
    )

    def normalize(col):
        return (col - col.min()) / (col.max() - col.min() + 1e-9)

    # Normalizações comuns
    summary['revenue_loss_mean'] = np.log1p(summary['revenue_loss_mean'])
    summary['miss_item_weighted_norm'] = normalize(summary['miss_item_weighted_r
    summary['revenue_loss_mean_norm'] = normalize(summary['revenue_loss_mean'])
    summary['rev_loss_weighted_norm'] = normalize(summary['rev_loss_weighted_rat
    summary['volume_norm'] = normalize(summary['total_orders'])

    if not long_format:

        summary['incidence_norm'] = normalize(summary['incidence_rate'])

        summary['risk_score'] = (
            summary['incidence_norm'] * 0.15 +
            summary['miss_item_weighted_norm'] * 0.15 +
            summary['revenue_loss_mean_norm'] * 0.35 +
            summary['rev_loss_weighted_norm'] * 0.25 +
            summary['volume_norm'] * 0.10
        )

    else:

        summary['risk_score'] = (
            summary['miss_item_weighted_norm'] * 0.30 +
            summary['revenue_loss_mean_norm'] * 0.35 +
            summary['rev_loss_weighted_norm'] * 0.25 +
            summary['volume_norm'] * 0.10
        )

    summary['risk_score_0_100'] = summary['risk_score'] * 100

```

```

summary['risk_level'] = pd.qcut(
    summary['risk_score_0_100'],
    q=4,
    labels=['Low Risk', 'Moderate Risk', 'High Risk', 'Critical Risk']
)

summary['risk_rank'] = summary['risk_score'].rank(ascending=False)

return summary.sort_values('risk_score_0_100', ascending=False)

# --/--

# -----
# 1 Radar Chart - Risk Profile
# -----
def plot_radar(df_risk, group_col, top_n=3, ax=None):

    if ax is None:
        fig, ax = plt.subplots(subplot_kw=dict(polar=True))

    df_top = df_risk.sort_values('risk_score_0_100', ascending=False).head(top_n)

    possible_dims = [
        'incidence_norm',
        'miss_item_weighted_norm',
        'revenue_loss_mean_norm',
        'rev_loss_weighted_norm',
        'volume_norm'
    ]

    dims = [d for d in possible_dims if d in df_risk.columns]

    angles = np.linspace(0, 2*np.pi, len(dims), endpoint=False).tolist()
    angles += angles[:1]

    for _, row in df_top.iterrows():

        values = row[dims].values.tolist()
        values += values[:1]

        ax.plot(angles, values, linewidth=2, label=row[group_col])
        ax.fill(angles, values, alpha=0.15)

    labels_map = {
        'incidence_norm': 'Missing Items Probability',
        'miss_item_weighted_norm': 'Weighted Missed Items',
        'revenue_loss_mean_norm': 'Average Revenue Loss',
        'rev_loss_weighted_norm': 'Weighted Revenue Loss',
        'volume_norm': 'Order Volume'
    }

    labels = [labels_map[d] for d in dims]

    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(labels)

    ax.set_title(f'Risk Profile - {group_col}')

    ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))

```

```

# -----
# 2 Heatmap - Risk by Dimension
# -----
def plot_heatmap(df_risk, group_col, ax=None):

    if ax is None:
        fig, ax = plt.subplots()

    possible_dims = [
        'incidence_norm',
        'miss_item_weighted_norm',
        'revenue_loss_mean_norm',
        'rev_loss_weighted_norm',
        'volume_norm'
    ]

    dims = [d for d in possible_dims if d in df_risk.columns]

    df_heat = df_risk.set_index(group_col)[dims]

    im = ax.imshow(df_heat, aspect='auto', cmap='viridis') # 🌈 melhor contraste

    labels_map = {
        'incidence_norm': 'Missing Items Probability',
        'miss_item_weighted_norm': 'Weighted Missed Items',
        'revenue_loss_mean_norm': 'Average Revenue Loss',
        'rev_loss_weighted_norm': 'Weighted Revenue Loss',
        'volume_norm': 'Order Volume'
    }

    labels = [labels_map[d] for d in dims]

    ax.set_xticks(range(len(dims)))
    ax.set_xticklabels(labels, rotation=45, ha='right', fontsize=10) # ✦ Legível
    ax.set_yticks(range(len(df_heat.index)))
    ax.set_yticklabels(df_heat.index, fontsize=9)

    cbar = plt.colorbar(im, ax=ax)
    cbar.set_label('Normalized Risk', fontsize=11)
    cbar.ax.tick_params(labelsize=10)

    ax.set_title(f'Risk Dimension Heatmap - {group_col}', fontsize=12)

    return im

# --/--

# -----
# Automated Fraud Risk Reporting
# -----
def generate_risk_reports(datasets, attributes, top_n_radar=3):

    reports = {}
    processed_attributes = set()

    for dataset_name, df in datasets.items():

        is_long = 'product_position' in df.columns

```

```

for attr in attributes:

    if attr in processed_attributes:
        continue

    if attr in df.columns:

        print("\n" + "="*50)
        print(f"Fraud Risk Analysis → {attr}")
        print("="*50 + "\n")

        df_risk = fraud_risk_framework(
            df,
            attr,
            long_format=is_long
        )

        reports[f'{dataset_name}_{attr}'] = df_risk

        # ♦ radar + heatmap lado a lado
        fig = plt.figure(figsize=(14,6))

        ax1 = fig.add_subplot(1,2,1, polar=True)
        ax2 = fig.add_subplot(1,2,2)

        plot_radar(df_risk, attr, top_n=top_n_radar, ax=ax1)
        plot_heatmap(df_risk, attr, ax=ax2)

        plt.tight_layout()
        plt.show()

        processed_attributes.add(attr)

    return reports

# --/--

# -----
# Final Combined High-Risk Table
# -----
def create_final_risk_table(risk_reports,
                            risk_levels=['Critical Risk','High Risk','Moderate R

    final_rows = []

    for key, df_risk in risk_reports.items():

        dataset, attr = key.split('_', 1)

        df_high = df_risk[df_risk['risk_level'].isin(risk_levels)].copy()

        if df_high.empty:
            continue

        df_high['dataset'] = dataset
        df_high['attribute'] = attr

        df_high['segment'] = df_high[attr]

        cols = [

```



```

        'dataset',
        'attribute',
        'segment',
        'total_orders',
        'incidence_rate',
        'miss_item_weighted_rate_%',
        'revenue_loss_mean',
        'rev_loss_weighted_rate_%',
        'risk_score',
        'risk_score_0_100',
        'risk_level'
    ]

    cols_existing = [c for c in cols if c in df_high.columns]

    final_rows.append(df_high[cols_existing])

    final_table = pd.concat(final_rows).reset_index(drop=True)

    final_table = final_table.sort_values('risk_score_0_100', ascending=False)

    return final_table

# --/--

# Top Fraud Risk Groups

def generate_fraud_groups(df_final, top_n=5):

    groups = df_final.head(top_n).copy()

    groups['fraud_driver'] = (
        groups['attribute'] + ' → ' + groups['segment'].astype(str)
    )

    return groups[['fraud_driver', 'risk_score_0_100', 'risk_level']]

# --/--

# Fraud Risk Leaderboard

def plot_risk_leaderboard(df_final, top_n=20):

    df_plot = df_final.head(top_n).copy()

    df_plot['label'] = df_plot['attribute'] + ' → ' + df_plot['segment'].astype(

    # color mapping
    color_map = {
        'Low Risk': 'green',
        'Moderate Risk': 'yellow',
        'High Risk': 'orange',
        'Critical Risk': 'red'
    }

    colors = df_plot['risk_level'].map(color_map)

    plt.figure(figsize=(12,6))

    plt.barh(
        df_plot['label'],

```

```

        df_plot['risk_score_0_100'],
        color=colors
    )

    plt.xlabel('Fraud Risk Score')
    plt.title('Top Fraud Risk Segments')

    plt.gca().invert_yaxis()

    for i, v in enumerate(df_plot['risk_score_0_100']):
        plt.text(v + 1, i, f'{v:.1f}')

    legend_handles = [
        mpatches.Patch(color=color, label=label)
        for label, color in color_map.items()
    ]

    plt.legend(handles=legend_handles, title="Risk Level")

    plt.show()

# --/--

# Fraud Risk Matrix (Impact × Frequency)
def plot_fraud_risk_matrix(df_final, top_n=40):

    df_plot = df_final.head(top_n).copy()

    df_plot['label'] = df_plot['attribute'] + ' → ' + df_plot['segment'].astype(

    # Color mapping
    color_map = {
        'Low Risk': 'green',
        'Moderate Risk': 'gold',
        'High Risk': 'orange',
        'Critical Risk': 'red'
    }

    colors = df_plot['risk_level'].map(color_map)

    # Medians (quadrants)
    x_med = df_plot['incidence_rate'].median()
    y_med = df_plot['revenue_loss_mean'].median()

    plt.figure(figsize=(13,9))

    plt.scatter(
        df_plot['incidence_rate'],
        df_plot['revenue_loss_mean'],
        s=50 + df_plot['risk_score_0_100'] * 2,
        c=colors,
        alpha=0.5,
        edgecolors='black',
        linewidth=0.5
    )

    plt.yscale('log')

    critical = df_plot[df_plot['risk_level'] == 'Critical Risk']

```

```

for _, row in critical.iterrows():

    x = row['incidence_rate']
    y = row['revenue_loss_mean']

    plt.annotate(
        row['segment'],
        xy=(x, y), # ponto real
        xytext=(x + 0.005, y * 1.02), # 🔥 AQUI entra o xytext
        textcoords='data',
        fontsize=9,
        weight='bold',
        color='darkred',
        arrowprops=dict(
            arrowstyle='-',
            color='gray',
            lw=0.8
        )
    )

# Quadrant Lines
plt.axvline(x_med, linestyle='--', color='gray', alpha=0.7)
plt.axhline(y_med, linestyle='--', color='gray', alpha=0.7)

# Labels
plt.xlabel('Missing Item Frequency (Incidence Rate)')
plt.ylabel('Financial Impact (Log Scale)')
plt.title('Fraud Risk Matrix (Impact vs Frequency)')

quadrant_text = (
    "Quadrants:\n"
    "Top Right → Highest Risk\n"
    "Top Left → High Impact\n"
    "Bottom Right → High Frequency\n"
    "Bottom Left → Low Priority"
)

plt.text(
    0.98, 0.02,
    quadrant_text,
    transform=plt.gca().transAxes,
    fontsize=9,
    ha='right',
    va='bottom',
    bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.9)
)

plt.grid(alpha=0.2)

plt.show()

# --/--

# =====
# Exporting Datasets
# =====

def export_eda_to_excel(file_name, missing=None, num_summary=None, cat_summary=None):

    with pd.ExcelWriter(file_name) as writer:

```

```

if missing is not None and not missing.empty:
    missing.to_excel(writer, sheet_name='Missing', index=False)

if num_summary is not None and not num_summary.empty:
    num_summary.to_excel(writer, sheet_name='Numerical', index=False)

if cat_summary is not None:
    cat_summary.to_excel(writer, sheet_name='Categorical', index=False)

print(f'\n Report exported to {file_name}')

```

4.0 Data Description

4.1 Loading data

```

In [98]: # Reading data on Windows
data_path = 'C:/Users/Igor/Repos/Walmart-Delivery-Fraud-Detection/datasets/'

df_orders_raw = pd.read_csv (data_path + 'orders.csv', low_memory=False)
df_missing_items_raw = pd.read_csv (data_path + 'missing_items_data.csv', low_me
df_products_raw = pd.read_csv (data_path + 'products_data.csv', low_memory=False)
df_customers_raw = pd.read_csv (data_path + 'customers_data.csv', low_memory=Fa
df_drivers_raw = pd.read_csv (data_path + 'drivers_data.csv', low_memory=False)

# reading data on Mac
# from pathlib import Path

# data_path_1 = Path('/Users/igorazevedo/Desktop/Walmart-Delivery-Fraud-Detectio

# df_orders_raw = pd.read_csv (data_path_1 / 'orders.csv', low_memory=False)
# df_missing_items_raw = pd.read_csv (data_path_1 / 'missing_items_data.csv', Lo
# df_products_raw = pd.read_csv (data_path_1 / 'products_data.csv', low_memory=F
# df_customers_raw = pd.read_csv (data_path_1 / 'customers_data.csv', low_memory
# df_drivers_raw = pd.read_csv (data_path_1 / 'drivers_data.csv', low_memory=Fa

```

```
In [99]: df_orders_raw.sample()
```

```
Out[99]:
```

	date	order_id	order_amount	region	items_delivered	items_missing
4865	30/06/2023	69776cc9-d1d2-475a-b0dc-556b6a9e059b	\$402.84	Sanford	16	0



```
In [100... df_missing_items_raw.sample()
```

```
Out[100...
```

	order_id	product_id_1	product_id_2	product_id_3
885	4b8c92f1-ec2b-4343-8501-f0ec804d0a40	PWPX0982761090984	NaN	NaN

In [101... `df_products_raw.sample()`

Out[101...

	produc_id	product_name	category	price
234	PWPX0982761091141	Pop-Tarts	Personal Care	\$6.00

In [102... `df_customers_raw.sample()`

Out[102...

	customer_id	customer_name	customer_age
567	WCID5503	Marvin Flowers	79

In [103... `df_drivers_raw.sample()`

Out[103...

	driver_id	driver_name	age	Trips
879	WDID10751	Christopher Weiss	41	54

In [104...

```
df_orders_1 = df_orders_raw.copy()
df_missing_items_1 = df_missing_items_raw.copy()
df_products_1 = df_products_raw.copy()
df_customers_1 = df_customers_raw.copy()
df_drivers_1 = df_drivers_raw.copy()
```

4.2 Columns Rename

In [105...

```
print(df_orders_1.columns, '\n')
print(df_missing_items_1.columns, '\n')
print(df_products_1.columns, '\n')
print(df_customers_1.columns, '\n')
print(df_drivers_1.columns)
```

```
Index(['date', 'order_id', 'order_amount', 'region', 'items_delivered',
      'items_missing', 'delivery_hour', 'driver_id', 'customer_id'],
      dtype='object')
```

```
Index(['order_id', 'product_id_1', 'product_id_2', 'product_id_3'], dtype='object')
```

```
Index(['produc_id', 'product_name', 'category', 'price'], dtype='object')
```

```
Index(['customer_id', 'customer_name', 'customer_age'], dtype='object')
```

```
Index(['driver_id', 'driver_name', 'age', 'Trips'], dtype='object')
```

Columns that need to be renamed:

Dataset `df_products_1` - 'produc_id', 'category', 'price'.

Dataset `df_missing_items` - 'product_id_1', 'product_id_2', 'product_id_3'.

Dataset `df_drivers` - 'age' and 'Trips'.

```
In [106... df_products_1 = df_products_1.rename(columns={'produc_id': 'product_id', 'catego
df_missing_items_1 = df_missing_items_1.rename(columns={'product_id_1': 'missing
df_drivers_1 = df_drivers_1.rename(columns={'age': 'driver_age', 'Trips': 'drive

In [107... print(df_products_1.columns, '\n')
print(df_missing_items_1.columns, '\n')
print(df_drivers_1.columns)
```

```
Index(['product_id', 'product_name', 'product_category', 'product_price'], dtype
='object')
```

```
Index(['order_id', 'missing_product_id_1', 'missing_product_id_2',
'missing_product_id_3'],
dtype='object')
```

```
Index(['driver_id', 'driver_name', 'driver_age', 'driver_trips'], dtype='object')
```

4.3 Data Dimensions

```
In [108... print(data_dimensions(df_orders_1, 'Orders Dataset'), '\n')
print(data_dimensions(df_missing_items_1, 'Missing Items Dataset'), '\n')
print(data_dimensions(df_products_1, 'Products Dataset'), '\n')
print(data_dimensions(df_customers_1, 'Customer Dataset'), '\n')
print(data_dimensions(df_drivers_1, 'Drivers Dataset'))
```

```
=== Data Dimensions Summary: Orders Dataset ===
```

```
Number of Rows: 10000
```

```
Number of Columns: 9
```

```
(10000, 9)
```

```
=== Data Dimensions Summary: Missing Items Dataset ===
```

```
Number of Rows: 1502
```

```
Number of Columns: 4
```

```
(1502, 4)
```

```
=== Data Dimensions Summary: Products Dataset ===
```

```
Number of Rows: 314
```

```
Number of Columns: 4
```

```
(314, 4)
```

```
=== Data Dimensions Summary: Customer Dataset ===
```

```
Number of Rows: 1239
```

```
Number of Columns: 3
```

```
(1239, 3)
```

```
=== Data Dimensions Summary: Drivers Dataset ===
```

```
Number of Rows: 1247
```

```
Number of Columns: 4
```

```
(1247, 4)
```

4.4 Data Types

4.4.1 Orders Dataset

In [109... df_orders_1.dtypes

```
Out[109...
date                object
order_id            object
order_amount        object
region             object
items_delivered     int64
items_missing       int64
delivery_hour       object
driver_id           object
customer_id         object
dtype: object
```

Necessary to update few columns types.

```
In [110...
# Before to update the column order_amount from object to float, it is necessary
df_orders_1['order_amount'] = (df_orders_1['order_amount'].str.replace('$', ''),

# Updating the colums type.
df_orders_1['date'] = pd.to_datetime( df_orders_1['date'].str.replace( '-' , '/' )
df_orders_1['delivery_hour'] = pd.to_datetime(df_orders_1['delivery_hour'],forma
df_orders_1['order_amount'] = df_orders_1['order_amount'].astype('float64')
df_orders_1['items_delivered'] = df_orders_1['items_delivered'].astype('int64')
df_orders_1['items_missing'] = df_orders_1['items_missing'].astype('int64')
```

In [111... df_orders_1.dtypes

```
Out[111...
date                datetime64[ns]
order_id            object
order_amount        float64
region             object
items_delivered     int64
items_missing       int64
delivery_hour       datetime64[ns]
driver_id           object
customer_id         object
dtype: object
```

4.4.2 Missing Items Dataset

In [112... df_missing_items_1.dtypes

```
Out[112...
order_id            object
missing_product_id_1 object
missing_product_id_2 object
missing_product_id_3 object
dtype: object
```

Not necessary to update the columns types.

4.4.3 Products Dataset

In [113... df_products_1.dtypes

```
Out[113...] product_id      object
           product_name  object
           product_category object
           product_price  object
           dtype: object
```

Necessary to update few columns types.

```
In [114...] # Before to update the column product_price from object to float, it is necessary
df_products_1['product_price'] = (df_products_1['product_price'].str.replace('$', ''))

# Updating the columns type.
df_products_1['product_price'] = df_products_1['product_price'].astype('float64')
```

```
In [115...] df_products_1.dtypes
```

```
Out[115...] product_id      object
           product_name  object
           product_category object
           product_price  float64
           dtype: object
```

4.4.4 Customers Dataset

```
In [116...] df_customers_1.dtypes
```

```
Out[116...] customer_id      object
           customer_name  object
           customer_age    int64
           dtype: object
```

Not necessary to update the columns types.

4.4.5 Drivers Dataset

```
In [117...] df_drivers_1.dtypes
```

```
Out[117...] driver_id      object
           driver_name  object
           driver_age    int64
           driver_trips  int64
           dtype: object
```

Not necessary to update the columns types.

4.5 Check NA

4.5.1 Orders Dataset

```
In [118...] missing_values_summary(df_orders_1, 'Orders')
```

```
=== Missing Values Summary: Orders ===
```


Out[118...

	index	missing_values	missing_%
0	date	0	0.0
1	order_id	0	0.0
2	order_amount	0	0.0
3	region	0	0.0
4	items_delivered	0	0.0
5	items_missing	0	0.0
6	delivery_hour	0	0.0
7	driver_id	0	0.0
8	customer_id	0	0.0

No missing values.

4.5.2 Missing Items Dataset

In [119...

```
missing_values_summary(df_missing_items_1, 'Missing Items')
```

```
=== Missing Values Summary: Missing Items ===
```

Out[119...

	index	missing_values	missing_%
3	missing_product_id_3	1484	98.801598
2	missing_product_id_2	1359	90.479361
1	missing_product_id_1	0	0.000000
0	order_id	0	0.000000

Missing values are expected because this dataframe lists the quantity of products not delivered, which variate between 1 to 3.

The columns product_id_1 2 and 3 lists the product id that was no delivered, if column product_id_2 and product_id_3 are set as NA, means that only one product is missing.

If only product_id_3 is set as NA, means that 2 products are missing.

If all products Ids columns have the Ids listed, means that 3 products are missing.

4.5.3 Produtcs Dataset

In [120...

```
missing_values_summary(df_products_1, 'Products')
```

```
=== Missing Values Summary: Products ===
```

Out[120...

	index	missing_values	missing_%
0	product_id	0	0.0
1	product_name	0	0.0
2	product_category	0	0.0
3	product_price	0	0.0

No Missing values

4.5.4 Customers Dataset

In [121...

```
missing_values_summary(df_customers_1, 'Customers')
```

```
=== Missing Values Summary: Customers ===
```

Out[121...

	index	missing_values	missing_%
0	customer_id	0	0.0
1	customer_name	0	0.0
2	customer_age	0	0.0

No Missing values

4.5.5 Drivers Dataset

In [122...

```
missing_values_summary(df_drivers_1, 'Drivers')
```

```
=== Missing Values Summary: Drivers ===
```

Out[122...

	index	missing_values	missing_%
0	driver_id	0	0.0
1	driver_name	0	0.0
2	driver_age	0	0.0
3	driver_trips	0	0.0

No Missing values

4.6 Check Unusual/Incoherent Values

4.6.1 Orders Dataset

In [123...

```
check_unusual_values(df_orders_1, numeric_cols=['order_amount', 'items_delivered'])
```

```
=====
Checking Unusual / Incoherent Values
=====
```

Numerical Columns:

Column: order_amount

Negative / Non-Numeroic values count: 0

Sample values: [20.08, 20.17, 20.17, 20.37, 20.41, 20.44, 20.45, 20.45, 20.46, 20.54, 20.61, '...', 1277.35, 1279.02, 1285.46, 1286.42, 1297.08, 1324.41, 1324.96, 1330.65, 1354.71, 1379.81, 1386.0]

Column: items_delivered

Negative / Non-Numeroic values count: 0

Sample values: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, '...', 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19]

Column: items_missing

Negative / Non-Numeroic values count: 0

Sample values: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, '...', 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]

Duplicate Checks:

Column: order_id → Duplicated values: 0 → Unique values match total rows: True

Categorical Columns:

Column: region

Unique values sample: ['Winter Park', 'Altamonte Springs', 'Clermont', 'Sanford', 'Apopka', 'Kissimmee', 'Orlando']

4.6.2 Missing Items Dataset

```
In [124... check_unusual_values( df_missing_items_1, id_cols=['order_id', 'missing_product_id_1', 'missing_product_id_2', 'missing_product_id_3'])
```

```
=====
Checking Unusual / Incoherent Values
=====
```

Duplicate Checks:

Column: order_id → Duplicated values: 0 → Unique values match total rows: True

Column: missing_product_id_1 → Duplicated values: 1357 → Unique values match total rows: False

Column: missing_product_id_2 → Duplicated values: 1398 → Unique values match total rows: False

Column: missing_product_id_3 → Duplicated values: 1483 → Unique values match total rows: False

Duplicates on columns 'product_id_1', 'product_id_2', 'product_id_3' are expected.

4.6.3 Products Dataset

```
In [125... check_unusual_values( df_products_1, numeric_cols=['product_price'], categorical_cols=['product_name', 'product_id_1', 'product_id_2', 'product_id_3'])
```

```
=====
Checking Unusual / Incoherent Values
=====
```

Numerical Columns:

Column: product_price

Negative / Non-Numeroic values count: 0

Sample values: [2.07, 2.08, 2.16, 2.21, 2.21, 2.31, 2.48, 2.62, 2.64, 2.65, 2.72, '...', 647.73, 690.88, 710.27, 725.5, 738.81, 750.72, 783.07, 808.21, 840.38, 880.38, 908.87]

Duplicate Checks:

Column: product_id → Duplicated values: 0 → Unique values match total rows: True

Column: product_name → Duplicated values: 0 → Unique values match total rows: True

Categorical Columns:

Column: product_category

Unique values sample: ['Supermarket', 'Bakery', 'Frozen', 'Beverages', 'Pantry', 'Dairy', 'Household', 'Snacks', 'Personal Care', 'Produce', 'Electronics']

4.6.4 Customers Dataset

```
In [126... check_unusual_values( df_customers_1, numeric_cols=['customer_age'], id_cols=['c
```

```
=====
Checking Unusual / Incoherent Values
=====
```

Numerical Columns:

Column: customer_age

Negative / Non-Numeroic values count: 0

Sample values: [18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, '...', 90, 90, 90, 90, 90, 90, 90, 90, 90]

Duplicate Checks:

Column: customer_id → Duplicated values: 0 → Unique values match total rows: True

Column: customer_name → Duplicated values: 9 → Unique values match total rows: False

4.6.4.1 Customers with duplicated names and ages

```
In [127... # df_customers_1[df_customers_1.duplicated(subset=['customer_name', 'customer_age'])]
# df_customers_1['customer_name'].value_counts()[lambda x: x > 1]

df_customers_1.groupby(['customer_name', 'customer_age']).size().reset_index(name='count')
df_customers_1.groupby(['customer_name', 'customer_age']).size() \
    .reset_index(name='count') \
    .query('count > 1')
```

Out[127... **customer_name customer_age count**

Despite there are duplicated customer names, they are not duplicated customer_id or customer_age, which means that they are different customers with the same name. Therefore, there is no inconsistency in the dataset.

4.6.5 Drivers Dataset

In [128... `check_unusual_values(df_drivers_1, numeric_cols=['driver_age', 'driver_trips'],`

```
=====
Checking Unusual / Incoherent Values
=====
```

Numerical Columns:

Column: driver_age

Negative / Non-Numeroic values count: 0

Sample values: [18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, '...', 65, 65, 65, 65, 65, 65, 65, 65, 65]

Column: driver_trips

Negative / Non-Numeroic values count: 0

Sample values: [11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, '...', 78, 78, 78, 78, 78, 78, 78, 78, 78]

Duplicate Checks:

Column: driver_id → Duplicated values: 0 → Unique values match total rows: True

Column: driver_name → Duplicated values: 18 → Unique values match total rows: False

4.6.5.1 Drivers with duplicated names and ages

```
In [129... # df_drivers_1[df_drivers_1.duplicated(subset=['driver_name', 'driver_age'], keep='first')
df_drivers_1.groupby(['driver_name', 'driver_age']).size().reset_index(name='count')
df_drivers_1.groupby(['driver_name', 'driver_age']).size() \
    .reset_index(name='count') \
    .query('count > 1')
```

Out[129... **driver_name driver_age count**

167	Brittany Morgan	19	2
295	Daniel Hall	20	2
945	Patricia Vance	18	2
1165	Timothy Brown	37	2

These duplicated driver names and ages could be the same person, but with different driver_id, which could be a indication of fraudulent activity, such as a driver using multiple identities to steal deliveries and earn more money.

Necessary to investigate further these duplicated driver names and ages, by checking if they have missing items on the orders they were responsible for.

5.0 Descriptive Statistics

```
In [130... df_orders_2 = df_orders_1.copy()
df_missing_items_2 = df_missing_items_1.copy()
df_products_2 = df_products_1.copy()
df_customers_2 = df_customers_1.copy()
df_drivers_2 = df_drivers_1.copy()
```

5.1 Orders Dataset

```
In [131... variables_summary(df_orders_2, dataset_name='Orders')
variables_summary_plots(df_orders_2)
```

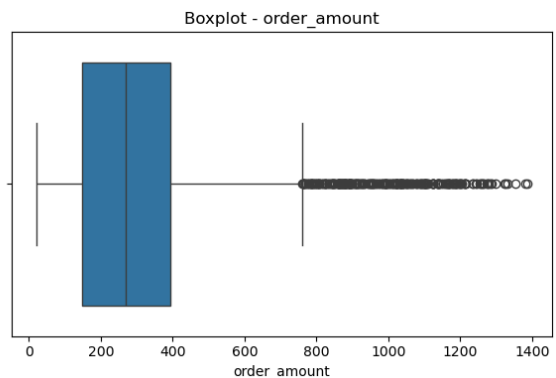
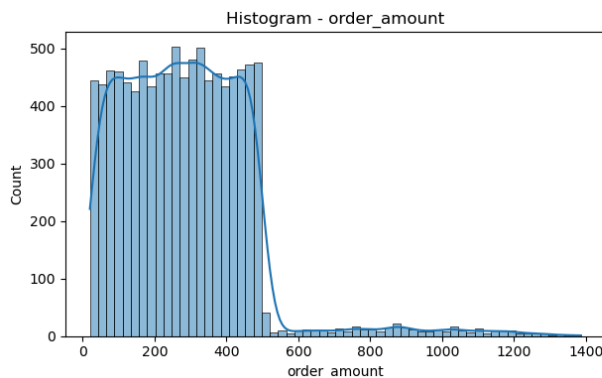
=====
Dataset - Orders
=====

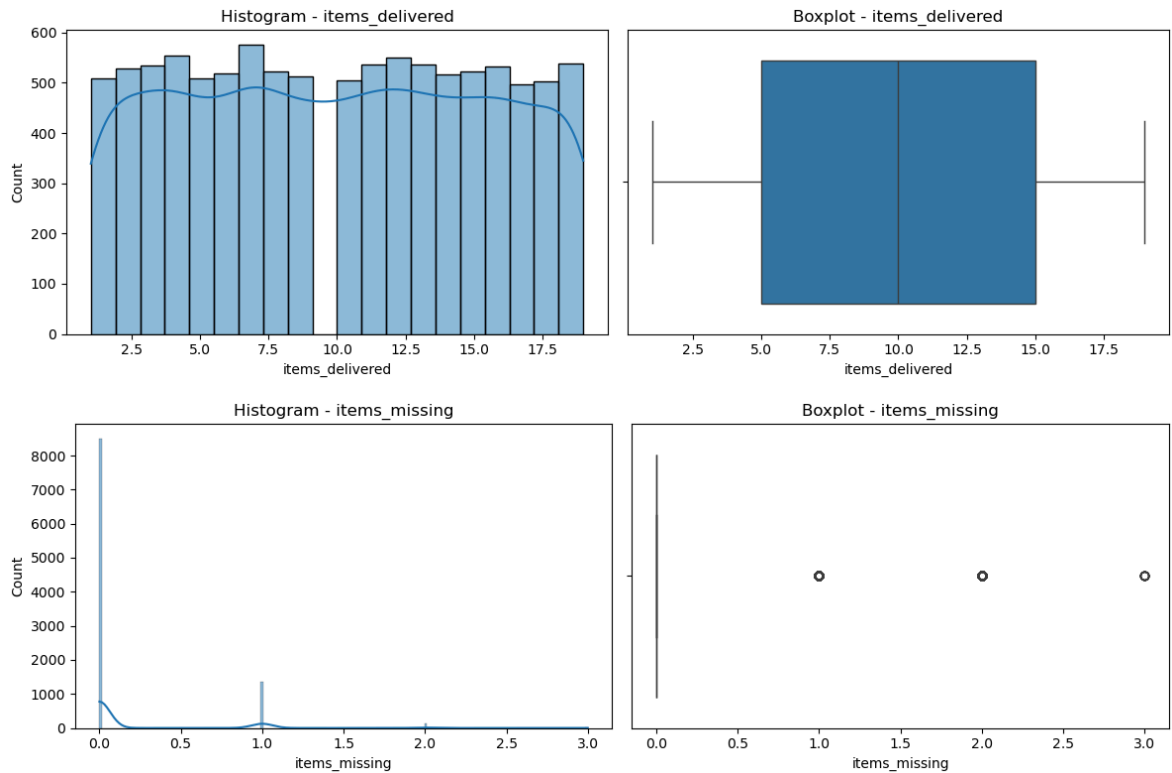
Numerical Summary:

	attribute	min	max	range	mean	median	std	skew
0	order_amount	20.08	1386.0	1365.92	283.302238	270.535	181.681638	1.547763
1	items_delivered	1.00	19.0	18.00	9.969400	10.000	5.459693	0.011747
2	items_missing	0.00	3.0	3.00	0.165700	0.000	0.414320	2.576987

Categorical Summary:

	attribute	unique_values	most_frequent	frequency
0	order_id	10000	00005d78-6f97-4699-a746-19826f415e38	1
1	region	7	Winter Park	1485
2	driver_id	1247	WDID09873	11
3	customer_id	1239	WCID6096	19





5.2 Missing Items Dataset

```
In [132... variables_summary(df_missing_items_2, dataset_name='Missing Items')
variables_summary_plots(df_missing_items_2)
```

Dataset - Missing Items

Categorical Summary:

	attribute	unique_values	most_frequent	frequency
0	order_id	1502	000e50be-92e8-46dc-9ff8-91a955b47ab7	1
1	missing_product_id_1	145	PWPX0982761090993	16
2	missing_product_id_2	103	PWPX0982761090982	2
3	missing_product_id_3	18	PWPX0982761090982	1

5.3 Products Dataset

```
In [133... variables_summary(df_products_2, dataset_name='Products')
variables_summary_plots(df_products_2)
```

Dataset - Products

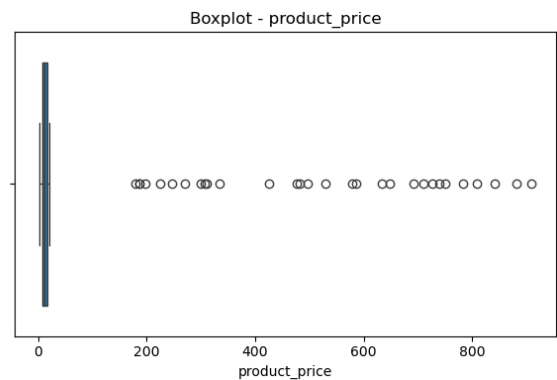
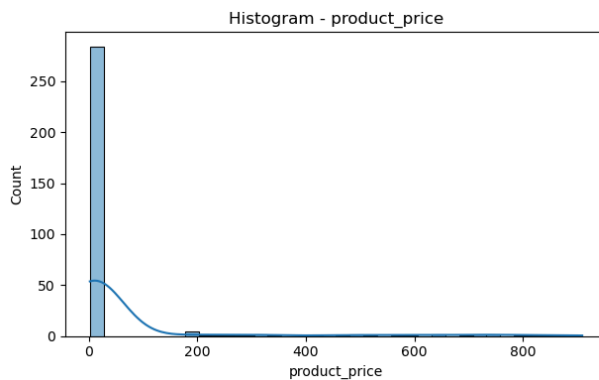
Numerical Summary:

	attribute	min	max	range	mean	median	std	skew	kurt
0	product_price	2.07	908.87	906.8	59.129936	11.275	164.990844	3.608371	12.285708



Categorical Summary:

	attribute	unique_values	most_frequent	frequency
0	product_id	314	PWPX0982761090982	1
1	product_name	314	7 Up	1
2	product_category	11	Supermarket	115



5.4 Customers Dataset

In [134...

```
variables_summary(df_customers_2, dataset_name='Customers')
variables_summary_plots(df_customers_2)
```

=====

Dataset - Customers

=====

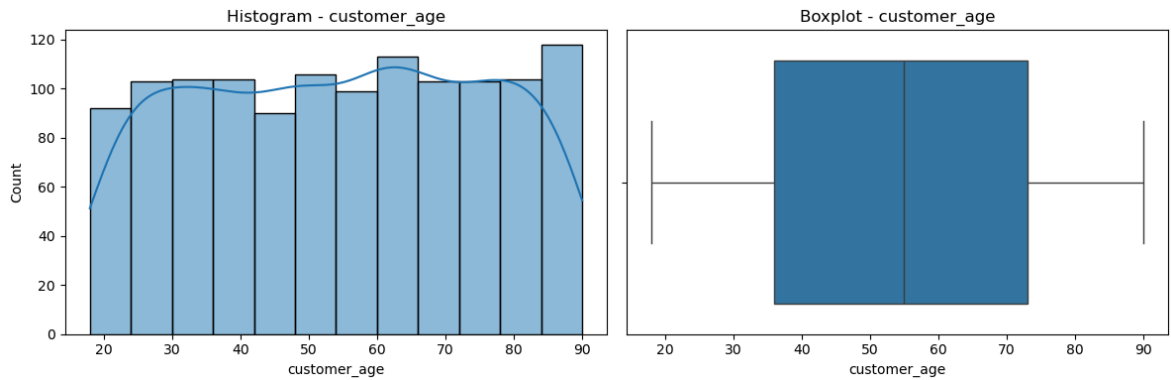
Numerical Summary:

	attribute	min	max	range	mean	median	std	skew	kurt
0	customer_age	18.0	90.0	72.0	54.421308	55.0	20.911501	-0.030328	-1.190029



Categorical Summary:

	attribute	unique_values	most_frequent	frequency
0	customer_id	1239	WCID5000	1
1	customer_name	1230	Angela Smith	2



5.5 Drivers Dataset

In [135...

```
variables_summary(df_drivers_2, dataset_name='Drivers')
variables_summary_plots(df_drivers_2)
```

=====

Dataset - Drivers

=====

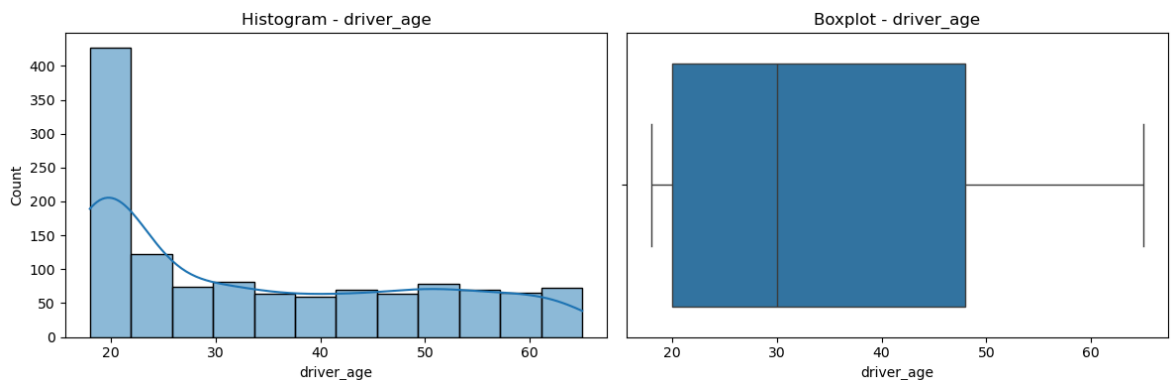
Numerical Summary:

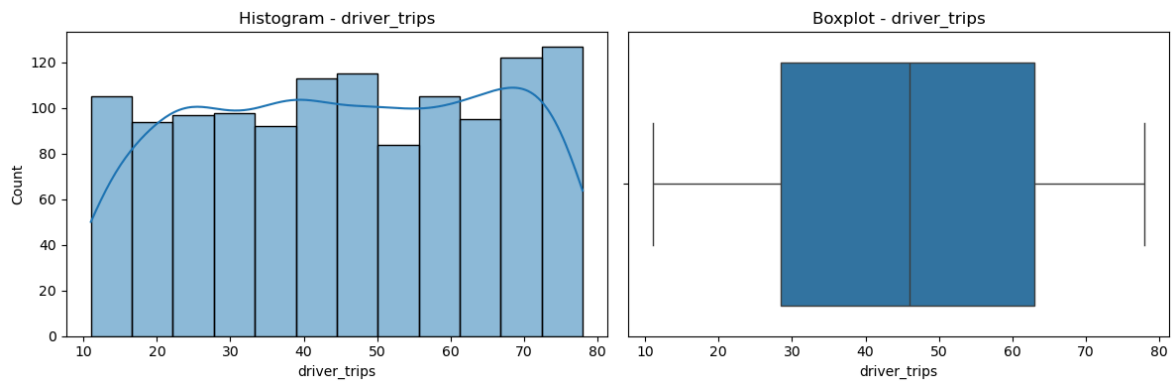
	attribute	min	max	range	mean	median	std	skew	kurt	out
0	driver_age	18.0	65.0	47.0	34.170810	30.0	15.441928	0.547450	-1.137527	
1	driver_trips	11.0	78.0	67.0	45.665597	46.0	19.747758	-0.049342	-1.203175	

◀ ▶

Categorical Summary:

	attribute	unique_values	most_frequent	frequency
0	driver_id	1247	WDID09872	1
1	driver_name	1229	Ashley Brown	2





5.6 Numerical Attributes – Distributional Behavior & Risk Implications

The numerical variables reveal distinct distributional patterns that may influence fraud risk detection and operational behavior.

Overall, monetary-related variables (e.g., order_amount and product_price) exhibit strong positive skewness and high kurtosis. This indicates a concentration of transactions at lower price levels combined with a small subset of extreme high-value observations.

Specifically:

Order Amount shows a wide range (20 to 1386), with most orders concentrated below \$500 but a heavy right tail driven by high-value purchases.

Product Price presents an even stronger disparity between median (11) and maximum (908), reinforcing the presence of extreme values likely associated with high-value product categories such as electronics.

This asymmetry suggests financial exposure is not evenly distributed across transactions. A small number of high-value orders may represent disproportionate operational and fraud risk.

In contrast, operational volume variables such as:

items_delivered

driver_trips

customer_age

driver_age

display approximately symmetric distributions with negative kurtosis (platykurtic behavior), indicating more evenly spread values and absence of heavy tails. These variables appear structurally stable and less influenced by extreme observations.

Note that for customer age, which there is a light positive skew, indicating marginal concentration of younger drivers, between 20 to 23 years.

The most critical variable from a fraud detection perspective is:

Items Missing

This attribute exhibits strong right skewness (2.57) and high kurtosis (6.92), with a clear zero-inflated pattern. Most transactions have no missing items, while a small fraction contains one or more missing units.

Although statistically flagged as outliers under IQR rules, values above zero represent meaningful operational events rather than noise and should not be removed.

This distribution suggests that fraud-related behavior, if present, is concentrated in a small subset of transactions rather than being widespread across the dataset.

Overall:

The dataset reveals structural asymmetry in financial variables and concentrated anomaly patterns in fulfillment behavior, suggesting that fraud risk is likely event-driven rather than systemic.

5.7 Categorical Attributes:

High-cardinality categorical variables such as `driver_id` and `customer_id` are not directly suitable for modeling and may require aggregation or encoding strategies.

6.0 Data Featuring

```
In [136... df_orders_3 = df_orders_2.copy()
df_missing_items_3 = df_missing_items_2.copy()
df_products_3 = df_products_2.copy()
df_customers_3 = df_customers_2.copy()
df_drivers_3 = df_drivers_2.copy()
```

6.1 Hypothesis Creation

6.1.1 Seasonality Hypothesis

H1: The missing item rates and revenue loss rates are significantly higher during night deliveries compared to morning and afternoon.

H2: Revenue loss rates and missing items rates increases during seasonal peak periods (e.g., summer and year-end).

H3: Missing item rates and revenue loss rates are higher on weekends compared to weekdays.

6.1.2 Location Hypothesis

H4: Missing item rates differ significantly across regions.

H5: The average monetary loss per missing item differs significantly across regions.

6.1.3 Products Hypothesis

H6: Electronics represent a disproportionately high share of missing items and revenue loss.

H7: Missing frequency and revenue loss are inversely related to product price.

H8 Within electronics, watches are more reported as missing item, while, within supermarket, beverages are more reported as missing item.

6.1.4 Customers Hypothesis

H9: Missing item rates and revenue loss rates differ significantly across customer age groups.

6.1.5 Drivers Hypothesis

H10: Drivers associated with multiple IDs exhibit higher missing item and revenue loss rates than drivers with a single ID.

H11: Missing item rates and revenue loss rates differ significantly across driver age groups.

H12: Missing item rate and Revenue Loss rate per trip increases with driver trip volume.

6.2 Additional Analysis (Operational Monitoring)

The following analysis highlights the top 15 customers and drivers with highest number of:

Missing Items reported Revenue loss generated

While not used for hypothesis validation, this analysis supports:

Fraud alert identification Operational monitoring Investigation prioritization

1. Top 15 Customers with highest number of items missing reported and revenue loss generated.
2. Top 15 Drivers with highest number of items missing reported and revenue loss generated.

6.3 Feature Engineering

6.3.1 Variables to be derivate from original variables

6.3.1.1 Month

```
In [137... df_orders_3['month'] = df_orders_3['date'].dt.month.astype('int64')
```

6.3.1.2 Weekday

```
In [138... df_orders_3['day_of_week'] = df_orders_3['date'].dt.day_name()
```

6.3.1.3 Period Of The Day

```
In [139... df_orders_3['hour'] = df_orders_3['delivery_hour'].dt.hour.astype('int64')

def period_of_day(hour):
    if 0 <= hour < 6:
        return 'Late Night'
    elif 6 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 18:
        return 'Afternoon'
    else:
        return 'Night'

df_orders_3['period'] = df_orders_3['hour'].apply(period_of_day)
```

6.3.1.4 Macro Category

```
In [140... df_products_3['product_category'].value_counts()
```

```
Out[140... product_category
Supermarket      115
Electronics      30
Bakery           22
Snacks           21
Household        21
Dairy            20
Pantry           19
Frozen           18
Produce          18
Beverages        17
Personal Care    13
Name: count, dtype: int64
```

Aside the Eletronics, all other categories are sub-categories from the macro category Supermaket. Therefore, a category called 'macro_category' will be createed to separate eletronics from groceries.

```
In [141... df_products_3['macro_category'] = df_products_3['product_category'].apply(
    lambda x: 'Electronics' if x == 'Electronics' else 'Supermarket' )

## Moving the column 'macro_category' to the penultimate position.

## Remove the column from the end
```

```
col = df_products_3.pop('macro_category')
## Insert the column in the desired position (penultimate).
df_products_3.insert(len(df_products_3.columns)-1, 'macro_category', col)
```

6.3.1.5 Customers Age Group

```
In [142... bins_customer = [18, 25, 40, 60, 90]
labels_customer = ['Young Adult', 'Adult', 'Middle Age', 'Senior']

df_customers_3['customer_age_group'] = pd.cut(df_customers_3['customer_age'],
                                              bins=bins_customer, labels=labels_c
```

6.3.1.6 Drivers Age Group

```
In [143... bins_driver = [18, 25, 40, 55, 65]
labels_driver = ['Young', 'Adult', 'Experienced', 'Senior']

df_drivers_3['driver_age_group'] = pd.cut(df_drivers_3['driver_age'],
                                          bins=bins_driver, labels=labels_driver,
```

6.3.1.7 Drivers ID Type

Creating a variable to identify if the driver has more than one ID.

```
In [144... driver_duplicates = (df_drivers_3.groupby(['driver_name', 'driver_age']).size().r

driver_duplicates = driver_duplicates.query('id_count > 1')
driver_duplicates

df_drivers_3['driver_id_type'] = df_drivers_3.duplicated(subset=['driver_name',
                                                                keep=False).apply(lambda
```

6.3.1.8 Drivers Trip Bin

```
In [145... labels = ['Low (11-24 trips)', 'Medium (25-41 trips)', 'High (42-60 trips)', 'Ve

df_drivers_3['trip_bin'] = pd.qcut(df_drivers_3['driver_trips'], q=4, labels=lab
```

6.3.1.9 Missing Flag

This column identifies if a order have at least one missing item (1) or not (0).

It is used to verify the missing rate.

With this column, it is possible to calculate the probability of a order have at least one missing item.

" Whats is probability of a order have at least one missing item?"

This is occurrence rate.

```
In [146... df_orders_3['missing_flag'] = (df_orders_3['items_missing'] > 0).astype(int)
```

6.3.1.10 Revenue Loss

In order to create this variable, it is necessary to merge the datasets `df_orders_3`, `df_missing_items_3` and `df_products_3`.

```
In [147... df = pd.merge(df_orders_3, df_missing_items_3, on='order_id', how='left')
```

```
In [148... #Merging df_orders_3 with df_missing_items_3.
df = pd.merge(df_orders_3, df_missing_items_3, on='order_id', how='left')

# merging df with df_products_3.
df_merge_1 = df.copy()

# Suffix List (1, 2, 3).
for i in [1, 2, 3]:

    df_merge_1 = df_merge_1.merge(
        df_products_3,
        left_on=f'missing_product_id_{i}',
        right_on='product_id',
        how='left'
    )

# Renaming columns.
df_merge_1 = df_merge_1.rename(columns={
    'product_name': f'product_name_{i}',
    'product_category': f'product_category_{i}',
    'macro_category': f'macro_category_{i}',
    'product_price': f'product_price_{i}'
})

# Removing auxiliar column.
df_merge_1 = df_merge_1.drop(columns=['product_id'])

#Converting NAN values to 0 for the price columns
price_cols = [f'product_price_{i}' for i in [1, 2, 3]]
df_merge_1[price_cols] = df_merge_1[price_cols].fillna(0)

# Creating revenue loss variable.
df_merge_1['revenue_loss'] = df_merge_1[price_cols].sum(axis=1)

# Reordering columns.
base_cols = [
    'date', 'order_id', 'order_amount', 'region',
    'items_delivered', 'items_missing', 'missing_flag',
    'delivery_hour', 'driver_id', 'customer_id',
    'revenue_loss', 'month', 'day_of_week', 'hour', 'period'
]

product_cols = []

for i in [1, 2, 3]:
    product_cols.extend([
        f'missing_product_id_{i}',
        f'product_name_{i}',
        f'product_category_{i}',
        f'macro_category_{i}',
        f'product_price_{i}'
    ])

])
```

6.3.1.10.1 Revenue Loss validation by items missing

Checking if the revenue loss variable is statistically coherent with the items missing variable, in ordder worlds,if this variable was correctly created and make mathematical sense.

```
In [149... # Variable description.

df_merge_1[['items_missing','revenue_loss']].describe()
```

```
Out[149...      items_missing  revenue_loss
count      10000.00000  10000.000000
mean         0.16570    14.938269
std          0.41432    88.696161
min          0.00000     0.000000
25%          0.00000     0.000000
50%          0.00000     0.000000
75%          0.00000     0.000000
max          3.00000    922.000000
```

```
In [150... # Basic Logical Validation.

df_merge_1[df_merge_1['items_missing'] == 0]['revenue_loss'].sum()
```

```
Out[150... np.float64(0.0)
```

```
In [151... # Statistical relationship between items_missing and revenue_loss.

df_merge_1[['items_missing','revenue_loss']].corr()
```

```
Out[151...      items_missing  revenue_loss
items_missing      1.000000      0.372755
revenue_loss       0.372755      1.000000
```

```
In [152... # Mean by items quantity.

df_merge_1.groupby('items_missing')['revenue_loss'].mean()
```

```
Out[152... items_missing
0         0.000000
1      101.170228
2       87.376412
3       37.170000
Name: revenue_loss, dtype: float64
```

```
In [153... # inconsistent Verfication.

df_merge_1[
```



```
(df_merge_1['items_missing'] > 0) &
(df_merge_1['revenue_loss'] == 0)
]
```

Out[153...

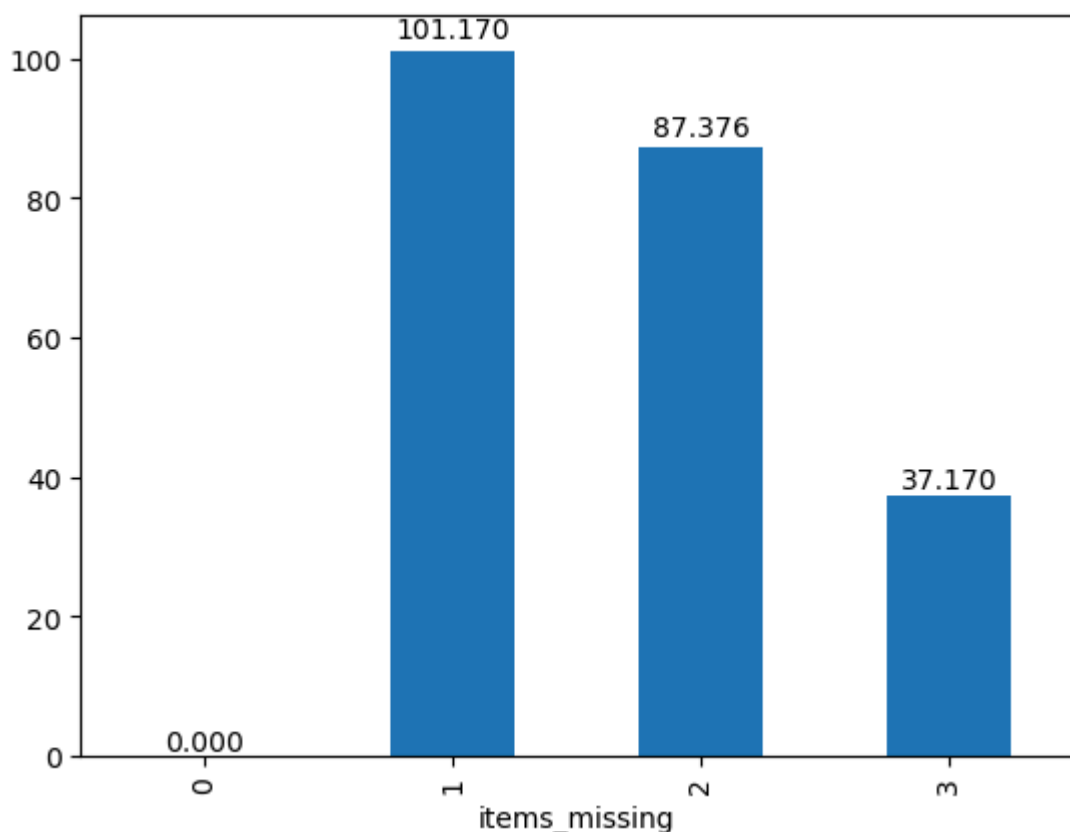
date	order_id	order_amount	region	items_delivered	items_missing	delivery_hour
------	----------	--------------	--------	-----------------	---------------	---------------

0 rows × 30 columns

In [154...

Growth Visualization.

```
plot_0 = df_merge_1.groupby('items_missing')['revenue_loss'].mean().plot(kind='bar')
bar_value(plot_0)
```



Revenue loss was validated to ensure structural consistency, showing zero loss for orders without missing items and a positive monotonic relationship with number of missing items."

7.0 Variables Filter

In order to know which sub-categories are significant for the analysis, it is necessary to verify the missing items per sub-category.

In [155...

```
print(df_merge_1.loc[df_merge_1['items_missing'] != 0, 'product_category_1'].val
print(df_merge_1.loc[df_merge_1['items_missing'] != 0, 'product_category_2'].val
print(df_merge_1.loc[df_merge_1['items_missing'] != 0, 'product_category_3'].val
```

```
product_category_1
Supermarket    1233
Electronics    269
Name: count, dtype: int64
```

```
product_category_2
Supermarket    143
Name: count, dtype: int64
```

```
product_category_3
Supermarket    18
Name: count, dtype: int64
```

In [156...

```
print(df_merge_1.groupby('product_category_1')['order_id'].count(), end='\n\n')
print(df_merge_1.groupby('product_category_2')['order_id'].count() , end='\n\n')
print(df_merge_1.groupby('product_category_3')['order_id'].count())
```

```
product_category_1
Electronics    269
Supermarket    1233
Name: order_id, dtype: int64
```

```
product_category_2
Supermarket    143
Name: order_id, dtype: int64
```

```
product_category_3
Supermarket    18
Name: order_id, dtype: int64
```

All the products that were not delivered are from the Supermarket category or Electronics category, therefore, it is not relevant to run an analysis by sub-category level, to verify if there is any sub-category that has a higher amount of missing products.

Additionally, there is no need to verify if the products are correctly categorized on the sub-category level.

8.0 Exploratory Data Analysis (EDA)

In [157...

```
df_orders_4 = df_orders_3.copy()
df_missing_items_4 = df_missing_items_3.copy()
df_products_4 = df_products_3.copy()
df_customers_4 = df_customers_3.copy()
df_drivers_4 = df_drivers_3.copy()
df_merge_ord_miss_prod = df_merge_1.copy()
```

8.1 Univariate Analysis

8.1.1 Response Variables

8.1.1.1 Items Missing

How many items were not delivered?

```

In [158... missing_counts = df_merge_ord_miss_prod['items_missing'].value_counts().sort_index
total_missing = df_merge_ord_miss_prod['items_missing'].sum()
total_delivered = df_merge_ord_miss_prod['items_delivered'].sum()

# Cálculo do percentual de perda/falta
total_general = total_delivered + total_missing
missing_pct = (total_missing / total_general) * 100

ax = missing_counts.plot(kind='bar')

ax.set_xlabel('Number of Missing Items')
ax.set_ylabel('Number of Orders')
ax.set_title('Distribution of Missing Items per Order')

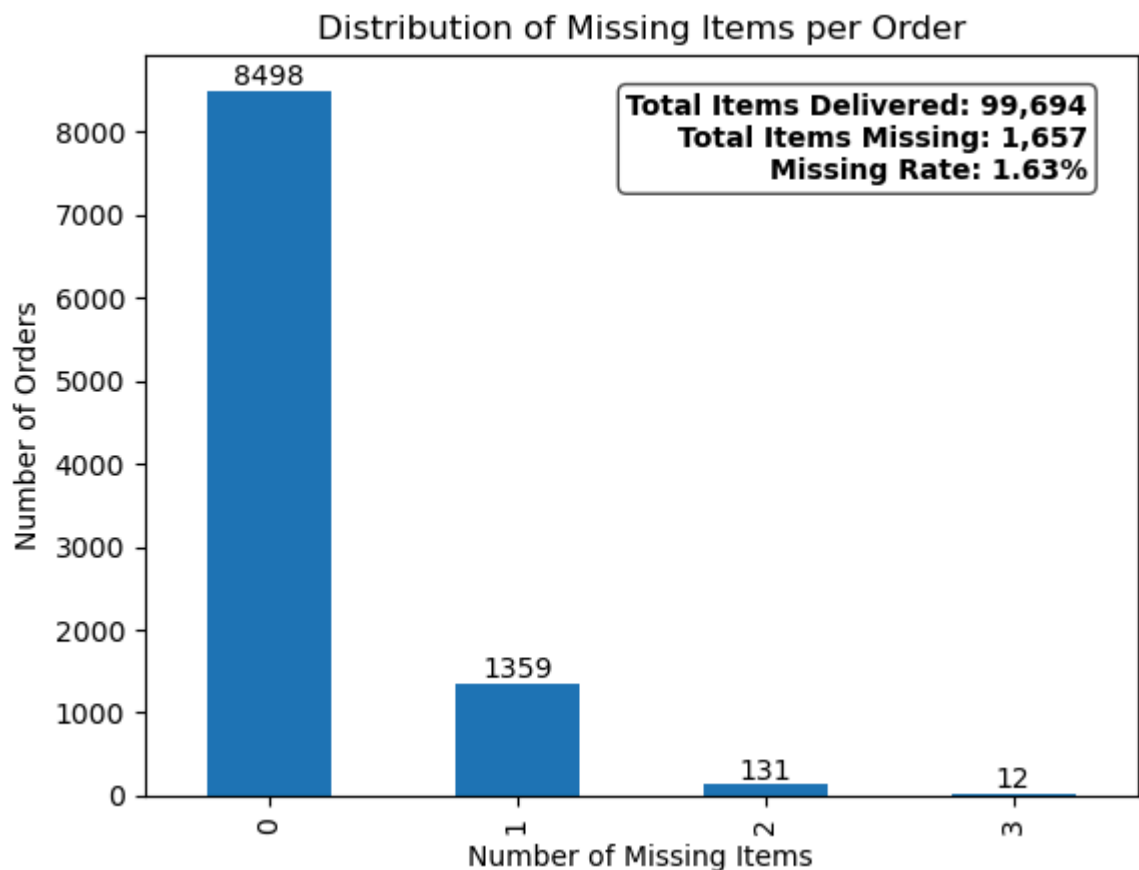
# Adiciona os valores no topo das barras
ax.bar_label(ax.containers[0])

# Criamos a string com as três informações
info_text = (f'Total Items Delivered: {total_delivered:,.0f}\n'
             f'Total Items Missing: {total_missing:,.0f}\n'
             f'Missing Rate: {missing_pct:.2f}%')

# Adiciona o bloco de texto no canto superior direito
ax.text(0.95, 0.95, info_text,
        transform=ax.transAxes,
        ha='right', va='top',
        fontsize=10,
        fontweight='bold', # Deixa o texto mais escuro/forte
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.7))

plt.show()

```



8.1.1.2 Revenue Loss

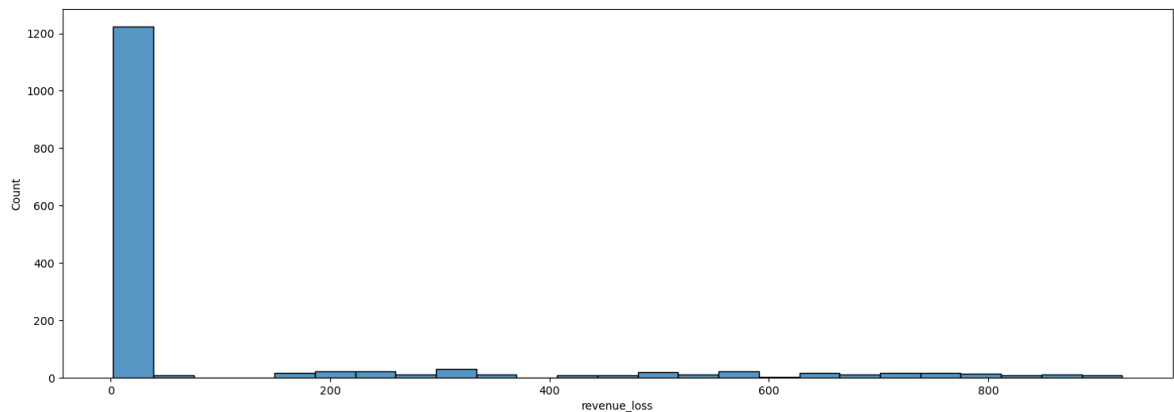
What is the total revenue lost due to these undelivered items?

```
In [159... df_res_var = df_merge_ord_miss_prod[df_merge_ord_miss_prod['items_missing'] > 0]

plt.figure(figsize=(18, 6))
sns.histplot(df_res_var['revenue_loss'], bins = 25)

print('The total of revenue loss is $', df_res_var['revenue_loss'].sum())
```

The total of revenue loss is \$ 149382.69



High skew and kurtosis. The variable revenue_loss has a high skew and kurtosis, which indicates that the distribution is not normal and has a long tail to the right. This suggests that there are some orders with very high revenue loss compared to the majority of orders, which may indicate that a small number of orders are responsible for a large portion of the total revenue loss.

```
In [160... total_revenue_loss = df_merge_ord_miss_prod['revenue_loss'].sum()
total_revenue = df_merge_ord_miss_prod['order_amount'].sum()
loss_percentage = (total_revenue_loss / total_revenue) * 100

fig = go.Figure(go.Indicator(
    mode="gauge+number",
    value=loss_percentage,

    number={
        'suffix': "%",
        'font': {'size': 45, 'color': '#2C3E50'}
    },

    title={
        'text': (
            "<b style='font-size:22px'>Revenue Overview</b><br>"
            "<span style='font-size:10px'>&nbsp;</span><br>"
            f"<b style='font-size:18px; color:#2C3E50'>Total Revenue: ${total_revenue}</b><br>"
            "<span style='font-size:8px'>&nbsp;</span><br>"
            f"<b style='font-size:18px; color:#2C3E50'>Revenue Loss: ${total_revenue_loss}</b>"
        )
    },

    gauge={
        'axis': {'range': [0,100]},
        'bar': {'color': "#E74C3C"},
```

```

        'bgcolor': "#2E86DE",
        'borderwidth': 0
    })
))

fig.update_layout(
    height=450,
    margin=dict(l=40, r=40, t=150, b=40)
)

fig.show()

```

8.2 Bivariate Analysis

8.2.1 Missing Incidence and Revenue Impact by Dimension

It is possible to calculate the probability of a order have at least one missing item (missing rate) using the column 'missing_flag'.

" Whats is probability of a order have at least one missing item?"

This is occurrence rate.

In [161...

```

# Calculating general average (general missing rate and revenue loss)
overall_incidence_missing_rate = df_merge_ord_miss_prod['missing_flag'].mean()
overall_revenue_loss = df_merge_ord_miss_prod['revenue_loss'].mean()

print("Overall Incidence Missing Rate:", overall_incidence_missing_rate, end='\n')
print("Overall Revenue Loss:", overall_revenue_loss)

```

Overall Incidence Missing Rate: 0.1502

Overall Revenue Loss: 14.938269

8.2.2.1 Seasonality

In [162...

```

# Calculating general avverage (general missing rate and revenue loss) per period

period_results = analyze_incidence_missing_rate(df_merge_ord_miss_prod, 'period')

order = ['Morning', 'Afternoon', 'Night', 'Late Night']

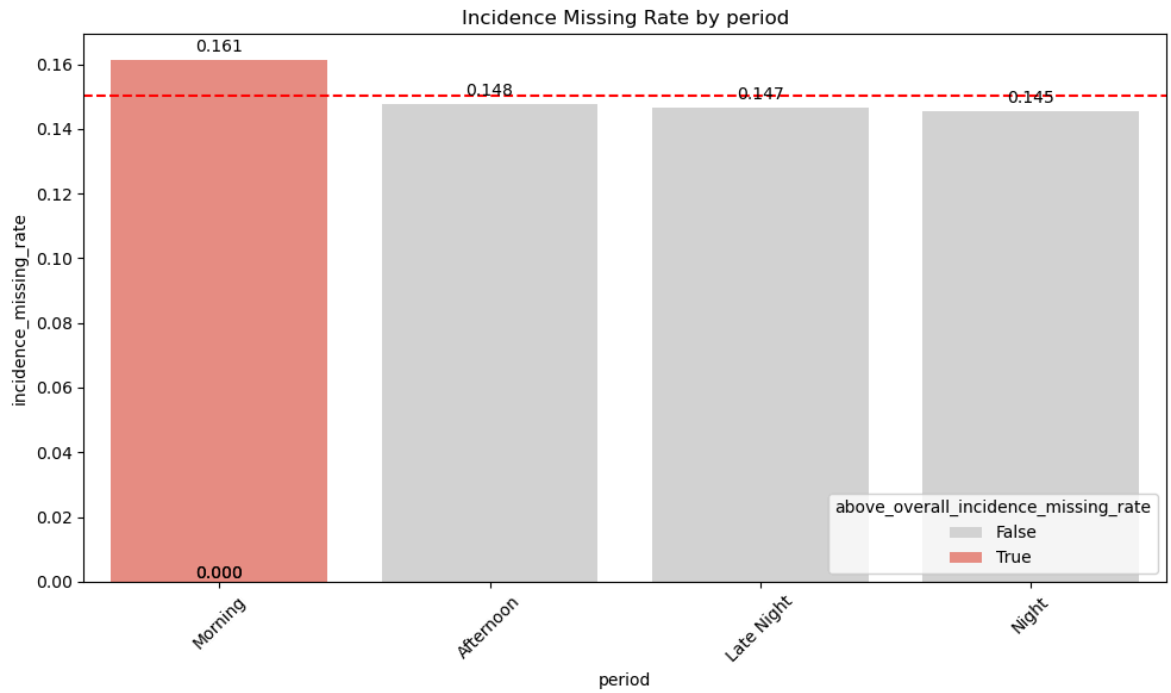
df['period'] = pd.Categorical(
    df['period'],
    categories=order,
    ordered=True
)
df_merge_1 = df_merge_1[base_cols + product_cols]
period_results

```

--- Analysis for: period ---

Total groups (after volume filter): 4

Groups above average: 1 (25.00%)



Out[162...

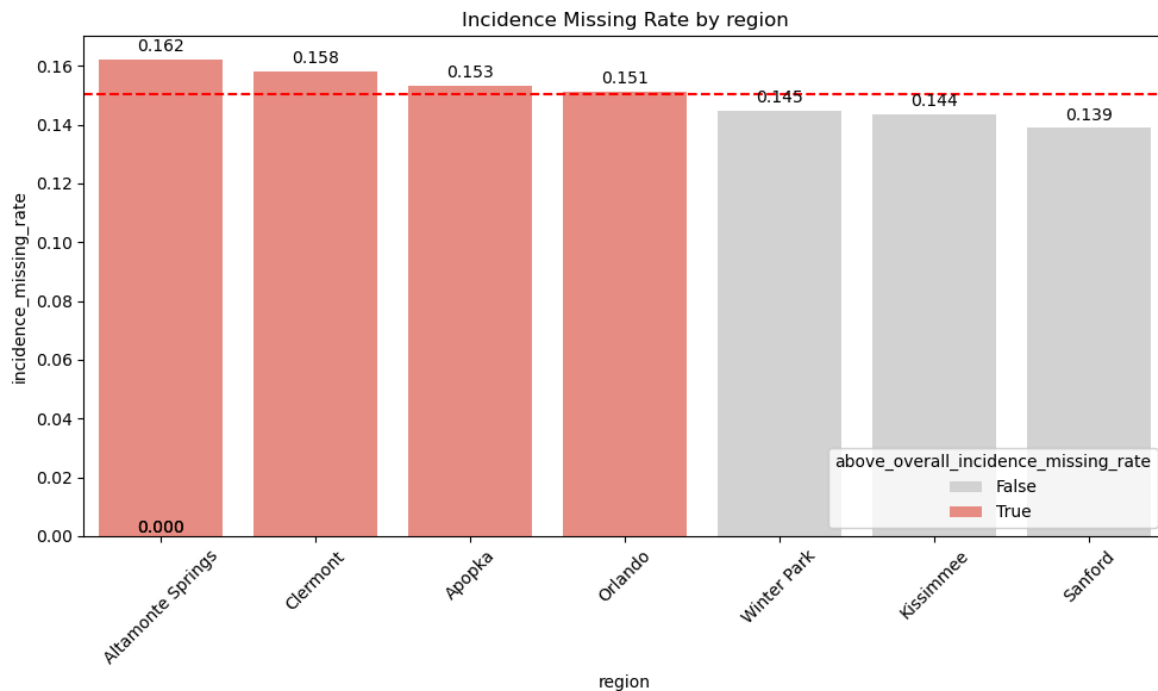
	period	incidence_missing_rate	avg_revenue_loss	total_orders	above_overall_incide
2	Morning	0.161225	15.885042	2481	
0	Afternoon	0.147664	15.041722	2526	
1	Late Night	0.146656	14.769194	2557	
3	Night	0.145320	14.044204	2436	

8.2.2.2 Location

In [163...

```
# Calculating general avverage (general missing rate and revenue loss) per regio
driver_results = analyze_incidence_missing_rate(df_merge_ord_miss_prod, 'region')
driver_results
```

```
--- Analysis for: region ---
Total groups (after volume filter): 7
Groups above average: 4 (57.14%)
```



Out[163...

	region	incidence_missing_rate	avg_revenue_loss	total_orders	above_overall_incid
0	Altamonte Springs	0.161992	16.512062	1426	
2	Clermont	0.158237	13.443945	1384	
1	Apopka	0.153305	13.439086	1422	
4	Orlando	0.151320	19.497873	1401	
6	Winter Park	0.144781	15.705589	1485	
3	Kissimmee	0.143561	13.459838	1421	
5	Sanford	0.138946	12.562587	1461	

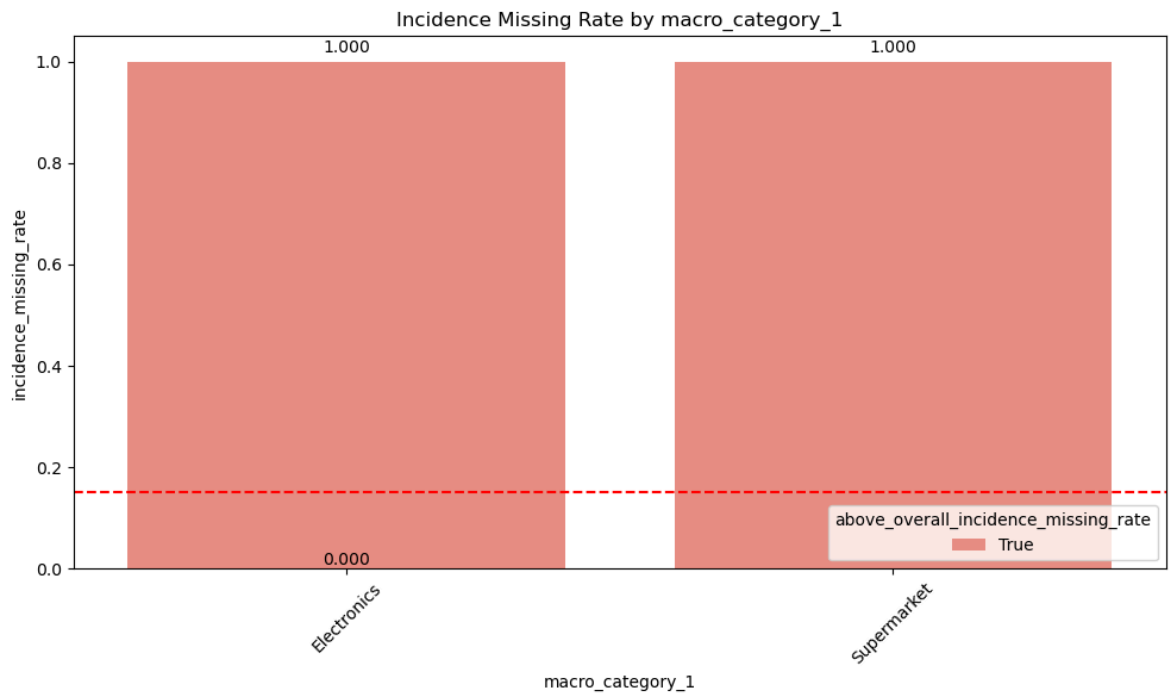
8.2.2.3 Product

In [164...

```
# Calculating general avverage (general missing rate and revenue loss) per macro
```

```
driver_results = analyze_incidence_missing_rate(df_merge_ord_miss_prod, 'macro_c
driver_results
```

```
--- Analysis for: macro_category_1 ---
Total groups (after volume filter): 2
Groups above average: 2 (100.00%)
```



Out[164...

	macro_category_1	incidence_missing_rate	avg_revenue_loss	total_orders	above_over
0	Electronics	1.0	496.537323	269	
1	Supermarket	1.0	12.825750	1233	

8.2.2.4 Customes

In [165...

```
# Calculating general avverage (general missing rate and revenue loss) per custo
driver_results = analyze_incidence_missing_rate(df_merge_ord_miss_prod, 'custome
driver_results
```

```
--- Analysis for: customer_id ---
Total groups (after volume filter): 358
Groups above average: 173 (48.32%)
```


Out[165...

	customer_id	incidence_missing_rate	avg_revenue_loss	total_orders	above_overall
685	WCID5685	0.583333	122.300833	12	
820	WCID5820	0.466667	6.745333	15	
7	WCID5007	0.428571	17.535000	14	
653	WCID5653	0.400000	130.511000	10	
817	WCID5817	0.400000	7.253000	10	
...	...	...	...	...	...
156	WCID5156	0.000000	0.000000	11	
98	WCID5098	0.000000	0.000000	13	
1156	WCID6156	0.000000	0.000000	11	
1174	WCID6174	0.000000	0.000000	11	
10	WCID5010	0.000000	0.000000	16	

358 rows × 6 columns



8.2.2.5 Drivers

In [166...

```
# Calculating general avverage (general missing rate and revenue loss) per drive
```

```
driver_results = analyze_incidence_missing_rate(df_merge_ord_miss_prod, 'driver_  
driver_results
```

```
--- Analysis for: driver_id ---  
Total groups (after volume filter): 425  
Groups above average: 425 (100.00%)
```

Out[166...

	driver_id	incidence_missing_rate	avg_revenue_loss	total_orders	above_overall_i
674	WDID10546	0.363636	65.236364	11	
673	WDID10545	0.363636	4.264545	11	
672	WDID10544	0.363636	69.580909	11	
193	WDID10065	0.363636	52.158182	11	
192	WDID10064	0.363636	21.345455	11	
...	...	...	...	...	...
804	WDID10676	0.272727	2.995455	11	
805	WDID10677	0.272727	74.866364	11	
806	WDID10678	0.272727	55.790000	11	
807	WDID10679	0.272727	4.150909	11	
808	WDID10680	0.272727	3.801818	11	

425 rows × 6 columns



8.2.2 Validating Hypothesis

H1: The missing item rates and revenue loss rates are significantly higher during night deliveries compared to morning and afternoon.

In [167...

```

df_h1 = hypothesis_metrics(df_merge_ord_miss_prod,'period',include_missing=True,
order_h1 = ['Morning', 'Afternoon', 'Night', 'Late Night'])
df_h1 = df_h1.sort_values(by='period',key=lambda x: pd.Categorical(x, categories

plt.figure(figsize=(18, 6))

# 1. Incidence Rate (Frequency of occurrence)
# Average of at Least one missing item per order.
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h1.set_index('period')['incidence_rate'].plot(kind='bar', ax=ax1,color='#4A90
df_h1.set_index('period')['revenue_loss_total'].plot(kind='line',ax=ax1_1,marker
ax1.set_title('Incidence Rate vs Total Revenue Loss by Period')
ax1.set_ylabel('Incidence Rate (At Least 1 Missing Item)')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

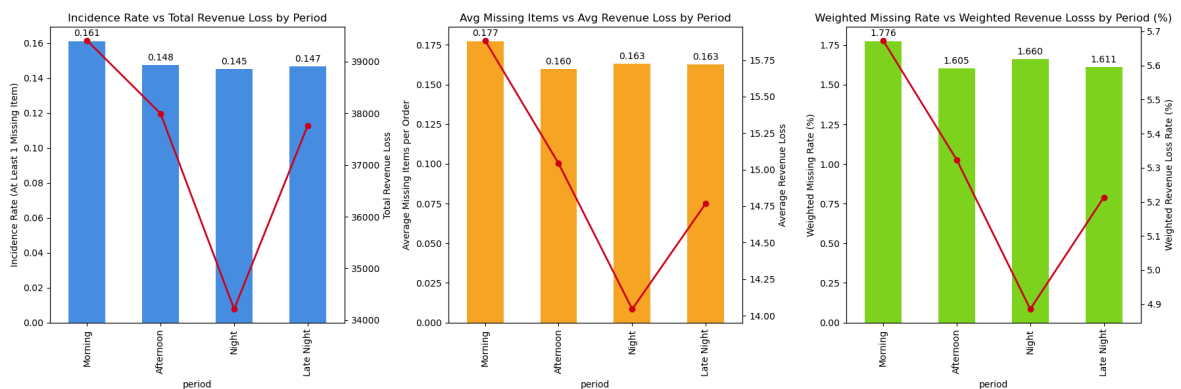
# 2. Average Missing Items (Error intensity)
# How many items, on average, are missing per order, regardless of how many item
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h1.set_index('period')['avg_missing'].plot(kind='bar',ax=ax2,color='#F5A623')
df_h1.set_index('period')['revenue_loss_mean'].plot(kind='line',ax=ax2_1,marker=
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by Period')
ax2.set_ylabel('Average Missing Items per Order')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

```

```
# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
# Average of missing items per delivered items - Sum of items missing divided by
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h1.set_index('period')['miss_item_weighted_rate_%'].plot(kind='bar',ax=ax3,co
df_h1.set_index('period')['rev_loss_weighted_rate_%'].plot(kind='line',ax=ax3_1,
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by Period (%)')
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h1)
```



Out[167...

	period	total_orders	incidence_rate	avg_missing	total_missing	std_missing	total
0	Morning	2481	0.1612	0.1773	440	0.424058	
1	Afternoon	2526	0.1477	0.1599	404	0.399696	
2	Night	2436	0.1453	0.1630	397	0.420374	
3	Late Night	2557	0.1467	0.1627	416	0.413164	

This hypothesis is not supported.

The Morning period presents the highest incidence rate (0.161), average missing items (0.177) and weighted missing rate (1.77%), as well the highest revenue losst average (15.88) and weighted missing rate (5.67%), indicating that missing items are both more frequent and more intense during this period.

Additionally, Morning shows the highest standard deviation (0.424), which is considerably higher than the average missing value. This suggests a high level of dispersion and the presence of extreme cases within this period, which may reflect either concentrated outliers or a broader variability of orders above the mean.

While this does not directly confirm fraud, the combination of higher rates and higher variability indicates greater operational instability and potential fraud risk exposure during the Morning period.

Although Late Night presents a similar total volume of missing items, its incidence and variability are lower than Morning, reinforcing Morning as the most critical operational window.

H2: Missing items and revenue loss rates increases during seasonal peak periods (e.g., summer and year-end).

```
In [168... df_h2 = hypothesis_metrics(df_merge_ord_miss_prod,'month',include_missing=True,i
order_h2 = [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
df_h2 = df_h2.sort_values(by='month',key=lambda x: pd.Categorical(x, categories=

plt.figure(figsize=(18, 6))

# 1. Revenue Loss total
ax1 = plt.subplot(3,1,1)
ax1_1 = ax1.twinx()
df_h2.set_index('month')['incidence_rate'].plot(kind='bar', ax=ax1,color='#4A90E
ax1_1.plot(range(len(df_h2)), df_h2['revenue_loss_total'], color='#D0021B', mark
ax1.set_title('Incidence Rate vs Total Revenue Loss by month')
ax1.set_ylabel('Incidence Rate')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

# 2. Revenue Loss avegare -
# Average of Revenue Loss per order, how much was the average of renevue loss pe
ax2 = plt.subplot(3,1,2)
ax2_1 = ax2.twinx()
df_h2.set_index('month')['avg_missing'].plot(kind='bar',ax=ax2,color='#F5A623')
ax2_1.plot(range(len(df_h2)), df_h2['revenue_loss_mean'], color='#D0021B', marke
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by month')
ax2.set_ylabel('Average Missing Items')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

# 3. Weighted Revenue Loss Rate (Weighted Rate - Real Impact).
# Average of revenue Loss per order amount - Sum of total revenue Loss by divide
ax3 = plt.subplot(3,1,3)
ax3_1 = ax3.twinx()
df_h2.set_index('month')['miss_item_weighted_rate_%'].plot(kind='bar',ax=ax3,col
ax3_1.plot(range(len(df_h2)), df_h2['rev_loss_weighted_rate_%'], color='#D0021B'
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by month (%)')
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h2)
```



Out[168...

	month	total_orders	incidence_rate	avg_missing	total_missing	std_missing	total_c
0	1	843	0.1459	0.1649	139	0.424992	
1	2	745	0.1342	0.1490	111	0.395645	
2	3	850	0.1671	0.1788	152	0.413008	
3	4	789	0.1534	0.1686	133	0.413264	
4	5	830	0.1699	0.1843	153	0.429318	
5	6	823	0.1373	0.1543	127	0.411812	
6	7	891	0.1571	0.1740	155	0.424047	
7	8	871	0.1561	0.1803	157	0.448113	
8	9	826	0.1392	0.1538	127	0.402226	
9	10	866	0.1397	0.1513	131	0.389432	
10	11	808	0.1646	0.1807	146	0.430582	
11	12	858	0.1364	0.1469	126	0.382671	

This hypothesis is not supported

Seasonal peak periods do not show a consistent increase in either missing item rates or revenue loss rates.

Missing item rates vary moderately across months, with the highest weighted missing rate observed in August (1.86%) and May (1.83%), rather than during typical peak periods such as summer or year-end.

Similarly, the highest revenue loss rates occur in the first quarter, particularly in March (6.96%), followed by January and February.

December presents relatively lower levels for both missing item rates and revenue loss.

Overall, the results suggest that operational risk and financial impact are not strongly driven by seasonal demand peaks, but instead fluctuate moderately throughout the year.

H3: Missing item rates and revenue loss rates are higher on weekends compared to weekdays.

In [169...

```

df_h3 = hypothesis_metrics(df_merge_ord_miss_prod, 'day_of_week', include_missing=
order_h3 = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',
df_h3 = df_h3.sort_values(by='day_of_week', key=lambda x: pd.Categorical(x, categ

plt.figure(figsize=(18, 6))

# 1. Incidence Rate (Frequency of occurrence)
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h3.set_index('day_of_week')['incidence_rate'].plot(kind='bar', ax=ax1, color='
df_h3.set_index('day_of_week')['revenue_loss_total'].plot(kind='line', ax=ax1_1, m
ax1.set_title('Incidence Rate vs Total Revenue Loss by day_of_week')
ax1.set_ylabel('Incidence Rate')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

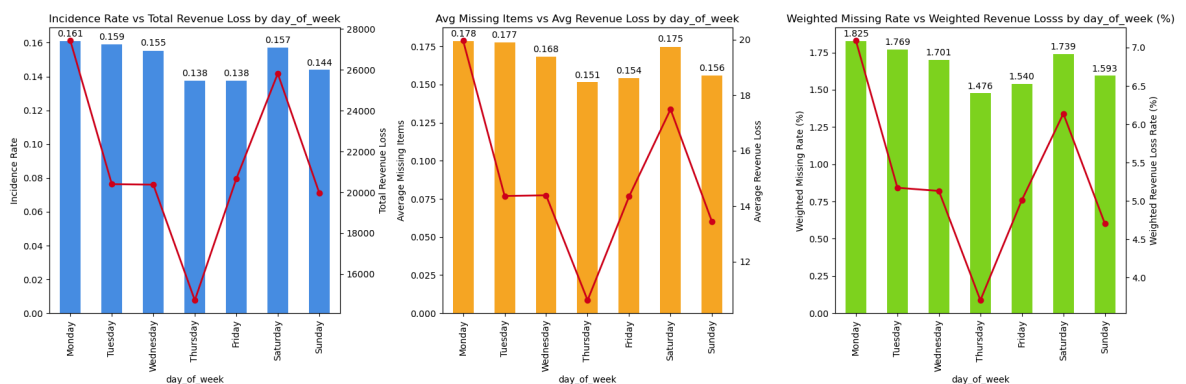
# 2. Average Missing Items (Error intensity)
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h3.set_index('day_of_week')['avg_missing'].plot(kind='bar', ax=ax2, color='#F5A
df_h3.set_index('day_of_week')['revenue_loss_mean'].plot(kind='line', ax=ax2_1, ma
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by day_of_week')
ax2.set_ylabel('Average Missing Items')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h3.set_index('day_of_week')['miss_item_weighted_rate_%'].plot(kind='bar', ax=a
df_h3.set_index('day_of_week')['rev_loss_weighted_rate_%'].plot(kind='line', ax=a
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by day_of_week (%)
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h3)

```



Out[169...

	day_of_week	total_orders	incidence_rate	avg_missing	total_missing	std_missing	tr
0	Monday	1374	0.1608	0.1783	245	0.427832	
1	Tuesday	1420	0.1592	0.1775	252	0.430742	
2	Wednesday	1416	0.1554	0.1681	238	0.410117	
3	Thursday	1387	0.1377	0.1514	210	0.396781	
4	Friday	1440	0.1375	0.1542	222	0.406493	
5	Saturday	1476	0.1572	0.1748	258	0.428559	
6	Sunday	1487	0.1439	0.1560	232	0.398351	

This hypothesis is not supported.

The hypothesis that weekend deliveries present higher missing item rates and revenue loss rates is not supported by the data.

The highest missing item rates are observed on Monday, with an incidence rate of 16.08% and a weighted missing rate of 1.83%. Tuesday and Wednesday also present relatively elevated values.

While Saturday shows comparatively high missing rates (1.74%), Sunday does not follow the same pattern, suggesting that the increase is not consistently associated with weekend operations.

A similar pattern is observed for financial impact. The highest revenue loss rate occurs on Monday (7.09%), followed by Saturday (6.14%). However, Sunday presents a moderate value, reinforcing the absence of a consistent weekend effect.

Overall, the results indicate that operational risk is more concentrated at the beginning of the week rather than during weekends, although Saturday remains a relevant day for monitoring due to its relatively high financial impact.

H4: Missing item rates differ significantly across regions.

In [170...

```

df_h4 = hypothesis_metrics(df_merge_ord_miss_prod, 'region', include_missing=True,
df_h4 = df_h4 = df_h4.sort_values(by='miss_item_weighted_rate_%', ascending=False)

plt.figure(figsize=(14, 10))

# 1. Incidence Rate (Frequency of occurrence)
ax1 = plt.subplot(3,1,1)
df_h4.set_index('region')['incidence_rate'].sort_values(ascending=True).plot(kind='bar',
ax1.set_title('Incidence Rate by Region')
bar_value(ax1, decimals=3, vertical=False)

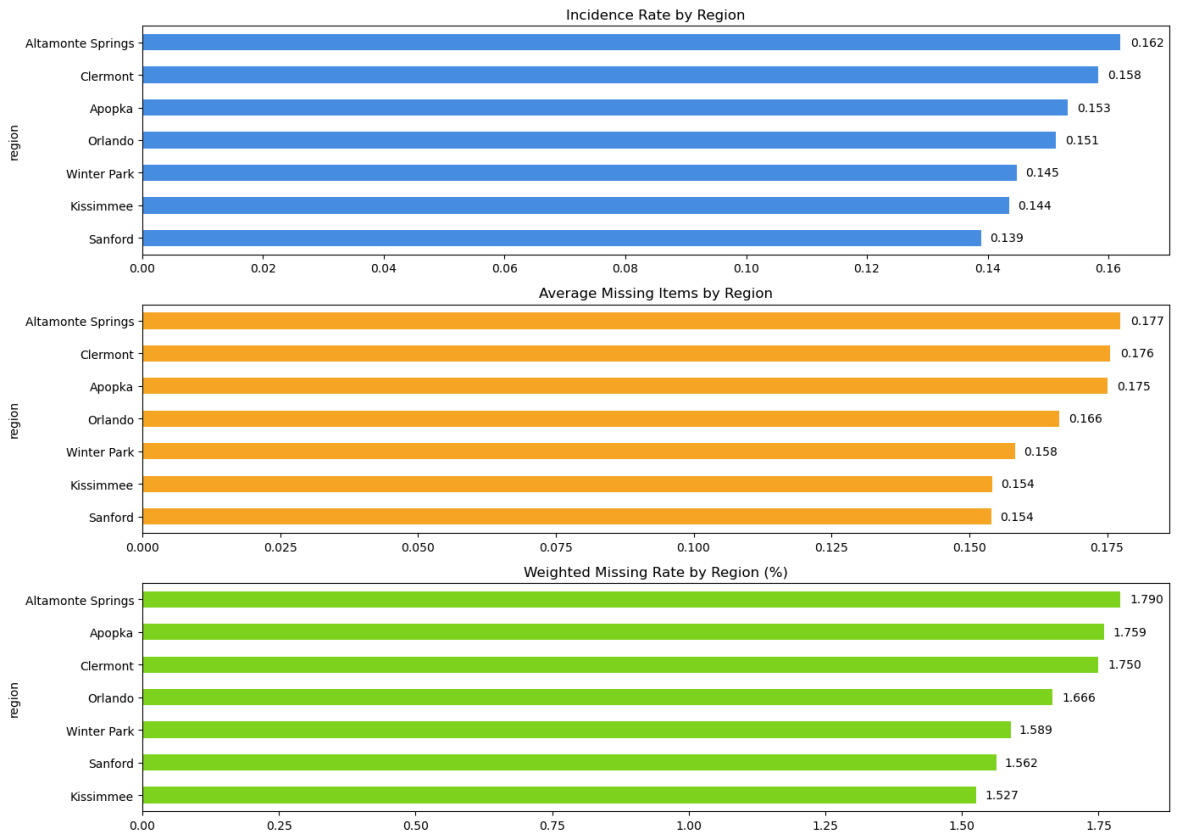
# 2. Average Missing Items (Error intensity)
ax2 = plt.subplot(3,1,2)
df_h4.set_index('region')['avg_missing'].sort_values(ascending=True).plot(kind='bar',
ax2.set_title('Average Missing Items by Region')
bar_value(ax2, decimals=3, vertical=False)

```

```
# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
ax3 = plt.subplot(3,1,3)
df_h4.set_index('region')['miss_item_weighted_rate_%'].sort_values(ascending=True)
ax3.set_title('Weighted Missing Rate by Region (%)')
bar_value(ax3, decimals=3, vertical=False)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h4)
```



Out[170...

	region	total_orders	incidence_rate	avg_missing	total_missing	std_missing	tota
0	Altamonte Springs	1426	0.1620	0.1774	253	0.423944	
1	Apopka	1422	0.1533	0.1751	249	0.438633	
2	Clermont	1384	0.1582	0.1756	243	0.425451	
3	Orlando	1401	0.1513	0.1663	233	0.414255	
4	Winter Park	1485	0.1448	0.1582	235	0.403665	
5	Sanford	1461	0.1389	0.1540	225	0.404046	
6	Kissimmee	1421	0.1436	0.1541	219	0.389337	

This hypothesis is partially supported. Moderately Supported (Weak-to-Moderate Evidence)

Altamonte Springs presents the highest incidence rate (16.20%), average missing items (0.177), and weighted missing rate (1.79%), indicating relatively higher operational risk compared to other regions.

Apopka and Clermont follow closely, also maintaining elevated rates across all key metrics.

On the lower end, Kissimmee and Sanford present the lowest weighted missing rates (1.53% and 1.56%, respectively), along with lower incidence and average missing levels.

However, the overall variation across regions remains moderate, with weighted rates ranging from 1.53% to 1.79% — a difference of approximately 0.26 percentage points. This suggests that while regional differences exist, they are not extreme, therefore the risk of fraud is relatively distributive.

Standard deviation values are relatively consistent across regions, indicating similar dispersion patterns in missing item distribution.

Overall, regional variability is present but not dramatically pronounced. Altamonte Springs, Apopka, and Clermont should be monitored more closely, although no region displays disproportionately abnormal behavior. There is a ranking but the difference is not strong enough to configurate critical regional risk. There is no significant evidence of anomaly the represents risk of fraude. The partner seems to be more operational than fraudulent.

H5: The average monetary loss per missing item differs significantly across regions.

In [171...

```
df_h5 = hypothesis_metrics(df_merge_ord_miss_prod, 'region', include_missing=False)
df_h5 = df_h5.sort_values(by='rev_loss_weighted_rate_%', ascending=False)

plt.figure(figsize=(14, 10))

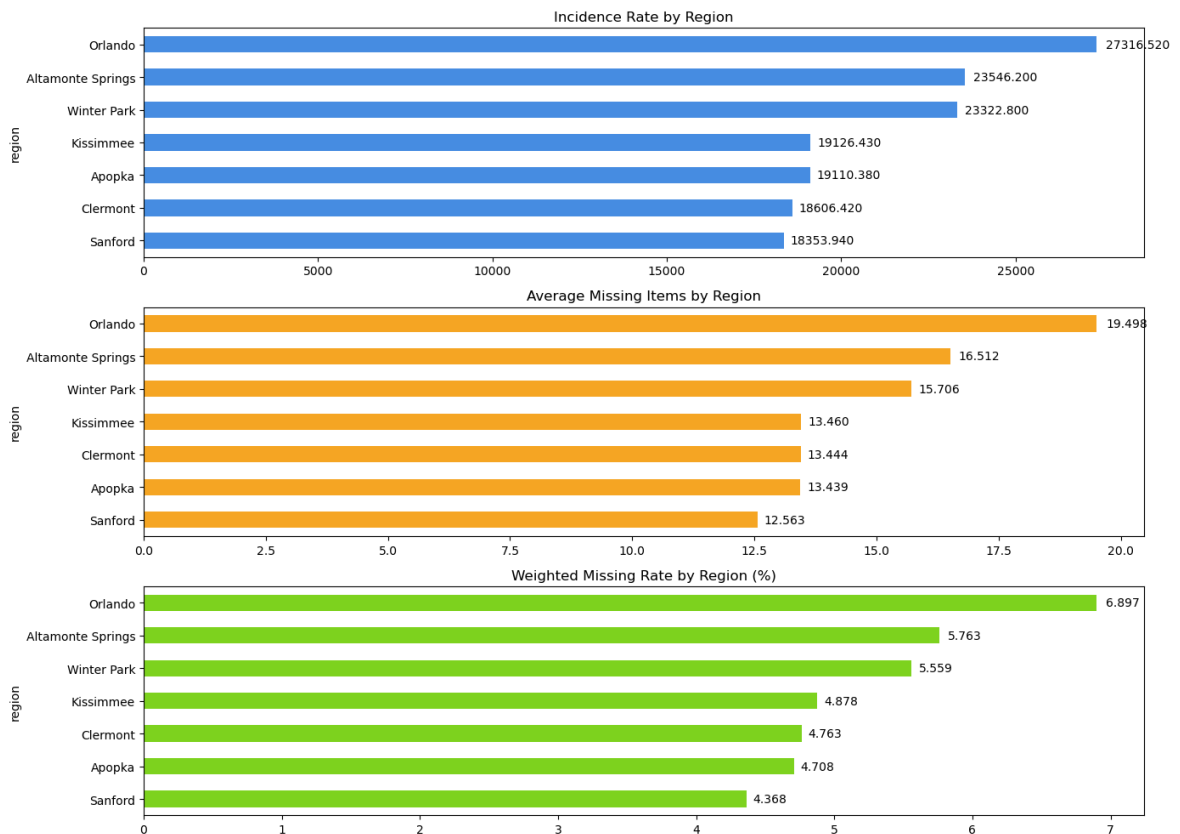
# 1. Revenue Loss total
ax1 = plt.subplot(3,1,1)
df_h5.set_index('region')['revenue_loss_total'].sort_values(ascending=True).plot()
ax1.set_title('Incidence Rate by Region')
bar_value(ax1, decimals=3, vertical=False)

# 2. Revenue Loss avegare
ax2 = plt.subplot(3,1,2)
df_h5.set_index('region')['revenue_loss_mean'].sort_values(ascending=True).plot()
ax2.set_title('Average Missing Items by Region')
bar_value(ax2, decimals=3, vertical=False)

# 3. Weighted Revenue Loss
ax3 = plt.subplot(3,1,3)
df_h5.set_index('region')['rev_loss_weighted_rate_%'].sort_values(ascending=True)
ax3.set_title('Weighted Missing Rate by Region (%)')
bar_value(ax3, decimals=3, vertical=False)

plt.tight_layout()
plt.show()
```

```
style_hypothesis_table(df_h5)
```



Out[171...

	region	total_orders	revenue_loss_mean	revenue_loss_total	total_order_amount
0	Orlando	1401	19.4979	27,316.5200	396044.150000
1	Altamonte Springs	1426	16.5121	23,546.2000	408562.150000
2	Winter Park	1485	15.7056	23,322.8000	419550.380000
3	Kissimmee	1421	13.4598	19,126.4300	392062.580000
4	Clermont	1384	13.4439	18,606.4200	390672.570000
5	Apopka	1422	13.4391	19,110.3800	405920.390000
6	Sanford	1461	12.5626	18,353.9400	420210.160000

This hypothesis is supported.

Missing items do not generate the same financial impact across regions. Orlando presents the highest average revenue loss per missing item (19.50), while Sanford shows the lowest (12.56), representing a difference of approximately 55%.

This suggests that missing items in Orlando tend to involve higher-value products, increasing the financial impact of fulfillment failures in that region.

H6 and H7:

For products hipphypothesis it is necessary to tranform the dataset from wide to long formart (one row per item missing).

```
In [172... # Selecting relevant columns
product_columns = [
    'order_id',
    'items_delivered',
    'items_missing',
    'order_amount',
    'missing_flag',
    'revenue_loss',

    # Product 1
    'missing_product_id_1', 'product_name_1',
    'product_category_1', 'macro_category_1', 'product_price_1',

    # Product 2
    'missing_product_id_2', 'product_name_2',
    'product_category_2', 'macro_category_2', 'product_price_2',

    # Product 3
    'missing_product_id_3', 'product_name_3',
    'product_category_3', 'macro_category_3', 'product_price_3'
]

df_products = df_merge_ord_miss_prod[product_columns].copy()

# Create a list to store the 3 datasets
dfs = []

for i in [1, 2, 3]:

    temp = df_products[[
        'order_id',
        'items_delivered',
        'items_missing',
        'order_amount',
        'missing_flag',
        'revenue_loss',
        f'missing_product_id_{i}',
        f'product_name_{i}',
        f'product_category_{i}',
        f'macro_category_{i}',
        f'product_price_{i}'
    ]].copy()

    temp.columns = [
        'order_id',
        'items_delivered',
        'items_missing',
        'order_amount',
        'missing_flag',
        'revenue_loss',
        'missing_product_id',
        'product_name',
        'product_category',
        'macro_category',
        'product_price'
    ]
```

```

temp['product_position'] = i

dfs.append(temp)

# Concatenate everything
df_products_long = pd.concat(dfs, ignore_index=True)

# Dropping NA values (removing rowws with no missing values)
df_products_long = df_products_long[df_products_long['macro_category'].notna()]

# Item missing indicator (column 'is_missing_item')
df_products_long['is_missing_item'] = df_products_long['missing_product_id'].notna()

# Bin Product Price
bins = [0,10,25,50,100,300,1000]

labels = [
    'Very Low ($0-10)',
    'Low ($10-25)',
    'Medium ($25-50)',
    'High ($50-100)',
    'Very High ($100-300)',
    'Premium ($300+)'
]

df_products_long['price_bin'] = pd.cut(
    df_products_long['product_price'],
    bins=bins,
    labels=labels
).astype(object)

```

H6: Electronics represent a disproportionately high share of missing items and revenue loss.

In [173...

```

df_h6 = (df_products_long.groupby('macro_category').agg(total_missing=('missing_item', 'sum'),
    revenue_loss_mean=('revenue_loss', 'mean'),
    revenue_loss_total=('revenue_loss', 'sum')))

# Missing per 1000 orders - Revenue
total_orders = df_merge_ord_miss_prod['order_id'].nunique()
df_h6['rev_loss_per_1000_orders'] = (df_h6['revenue_loss_total'] / total_orders)

# Missing item share - Percentage of itmes missing for the specific group
df_h6['missing_share_%'] = (df_h6['total_missing'] / df_h6['total_missing'].sum())

# Missing per 1000 orders - Items missing
df_h6['missing_per_1000_orders'] = (df_h6['total_missing'] / total_orders * 1000)

df_h6 = df_h6.sort_values('missing_share_%')
df_h6_1 = df_h6.reset_index()

plt.figure(figsize=(18,6))

# 1 Total missing
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h6['total_missing'].plot(kind='bar',ax=ax1,color='#4A90E2')
df_h6['revenue_loss_mean'].plot(kind='line',ax=ax1_1,marker='o',linewidth=2, col

```

```

ax1.set_title('Total Missing Items vs Average Revenue Loss by by Macro Category')
ax1.set_ylabel('Total Missing Items')
ax1_1.set_ylabel('Average Revenue Loss by')
bar_value(ax1, decimals=0)

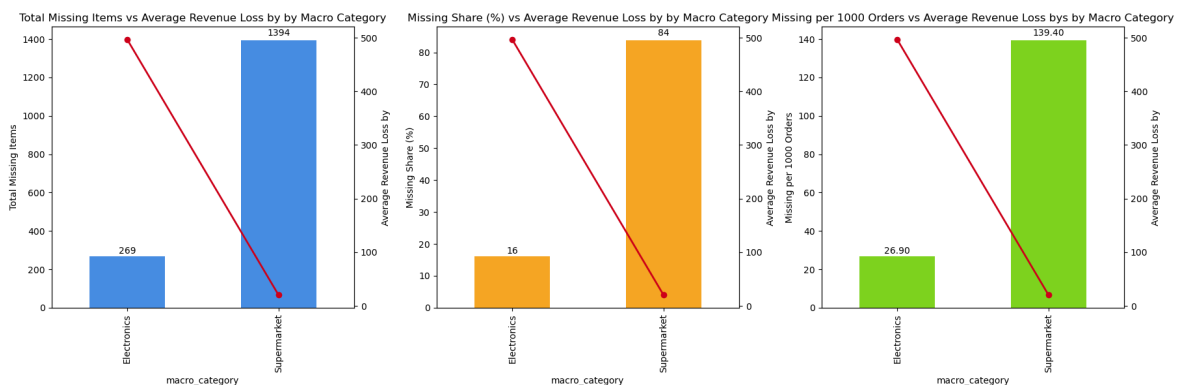
# # 2 Share - Percentage of itmes missing for the specific group
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h6['missing_share_%'].plot(kind='bar',ax=ax2,color='#F5A623')
df_h6['revenue_loss_mean'].plot(kind='line',ax=ax2_1,marker='o',linewidth=2, col
ax2.set_title('Missing Share (%) vs Average Revenue Loss by by Macro Category')
ax2.set_ylabel('Missing Share (%)')
ax2_1.set_ylabel('Average Revenue Loss by')
bar_value(ax2, decimals=0)

# 3 Missing per 1000 orders
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h6['missing_per_1000_orders'].plot(kind='bar',ax=ax3,color='#7ED321')
df_h6['revenue_loss_mean'].plot(kind='line',ax=ax3_1,marker='o',linewidth=2, col
ax3.set_title('Missing per 1000 Orders vs Average Revenue Loss bys by Macro Cate
ax3.set_ylabel('Missing per 1000 Orders')
ax3_1.set_ylabel('Average Revenue Loss by')
bar_value(ax3, decimals=2)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h6_1)

```



Out[173...

	macro_category	total_missing	revenue_loss_mean	revenue_loss_total	rev_loss_per_10
0	Electronics	269	496.5373	133,568.5400	1
1	Supermarket	1394	20.3557	28,375.8300	

This hypothesis is partially supported (supported for Revenue Loss, not supported for Missing Items).

The hypothesis that electronics represent a disproportionately high share of missing items and revenue loss is only partially supported by the data.

In terms of missing item volume, the supermarket category accounts for the vast majority of occurrences, representing 83.82% of all missing items and 139.4 missing items per

1,000 orders. Electronics represent a much smaller share, with 16.18% of missing items and 26.9 missing items per 1,000 orders.

However, the financial impact of missing items is substantially higher for electronics. The average revenue loss per incident reaches 496.54, compared to only 20.36 for supermarket items. As a result, electronics generate a total revenue loss of 133,568, nearly five times higher than the supermarket category despite having far fewer missing cases.

Overall, while supermarket items drive the majority of operational incidents, electronics account for the most significant financial risk due to their substantially higher unit value.

This insight is very powerful because it highlights a classic distinction in analytics:

Operational volume risk

Financial impact risk

H7: Missing frequency and revenue loss are inversely related to product price.

In [174...

```
df_h7 = (df_products_long.groupby('price_bin').agg(total_missing=('missing_flag',
                                                    avg_price=('product_price', 'm
                                                    revenue_loss_total=('revenue_
                                                    revenue_loss_mean=('revenue_

# Missing item share - Percentage of itmes missing for the specific group
df_h7['missing_share_%'] = (df_h7['total_missing'] / df_h7['total_missing'].sum())

# Missing per 1000 orders
total_orders = df_merge_ord_miss_prod['order_id'].nunique()
df_h7['missing_per_1000_orders'] = (df_h7['total_missing'] / total_orders) * 100

df_h7 = df_h7.sort_values('total_missing')
df_h7 = df_h7.reset_index()

plt.figure(figsize=(18,6))

# 1 Total missing
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h7.set_index('price_bin')['total_missing'].plot(kind='bar', ax=ax1, color='#4A9
df_h7.set_index('price_bin')['revenue_loss_mean'].plot(kind='line', ax=ax1_1, mark
ax1.set_title('Missing Items vs Average Revenue Loss by Price Range')
ax1.set_ylabel('Total Missing Items')
ax1_1.set_ylabel('Average Revenue Loss ($)')
bar_value(ax1, decimals=0)

# # 2 Share - Percentage of itmes missing for the specific group
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h7.set_index('price_bin')['missing_share_%'].plot(kind='bar', ax=ax2, color='#F
df_h7.set_index('price_bin')['revenue_loss_mean'].plot(kind='line', ax=ax2_1, mark
ax2.set_title('Missing Share (%) vs Average Revenue Loss by Price Range')
ax2.set_ylabel('Missing Share (%)')
ax2_1.set_ylabel('Average Revenue Loss ($)')
```

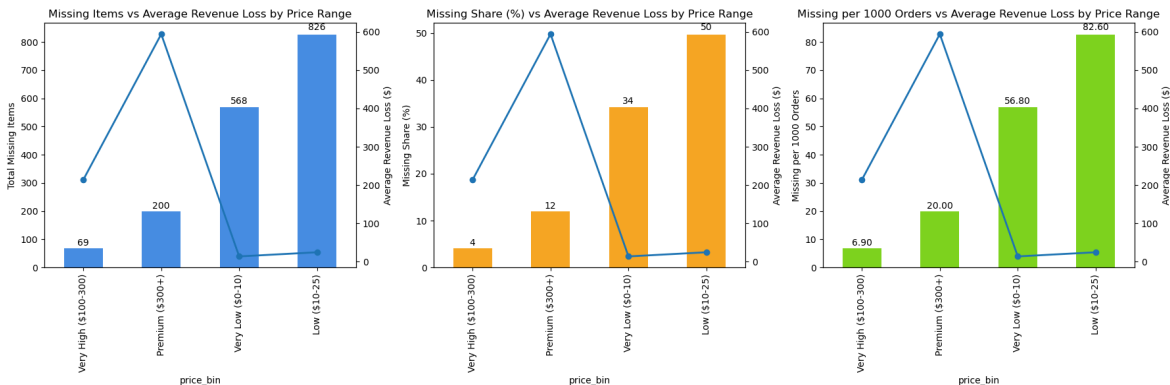
```

bar_value(ax2, decimals=0)

# 3 Missing per 1000 orders
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h7.set_index('price_bin')['missing_per_1000_orders'].plot(kind='bar',ax=ax3,color='red')
df_h7.set_index('price_bin')['revenue_loss_mean'].plot(kind='line',ax=ax3_1,marker='x',color='blue')
ax3.set_title('Missing per 1000 Orders vs Average Revenue Loss by Price Range')
ax3.set_ylabel('Missing per 1000 Orders')
ax3_1.set_ylabel('Average Revenue Loss ($)')
bar_value(ax3, decimals=2)

plt.tight_layout()
plt.show()

df_h7_1 = df_h7.sort_values('total_missing', ascending=False)
style_hypothesis_table(df_h7_1)
    
```



Out[174...

	price_bin	total_missing	avg_price	revenue_loss_total	revenue_loss_mean	missing_per_1000_orders
3	Low (\$10-25)	826	14.851283	20,454.5600	24.7634	82.60
2	Very Low (\$0-10)	568	6.543891	7,921.2700	13.9459	56.80
1	Premium (\$300+)	200	593.345650	118,803.8500	594.0193	20.00
0	Very High (\$100-300)	69	213.470580	14,764.6900	213.9810	6.90

This hypothesis is partially supported (not supported for Revenue Loss, supported for Missing Items).

The hypothesis that missing frequency and revenue loss are inversely related to product price is only partially supported by the data.

Lower-priced items show substantially higher missing frequencies. Products priced between 10–25 present the highest rate, reaching 82.6 missing items per 1,000 orders, followed by 10 with 56.8 per 1,000 orders.

In contrast, higher-priced items show significantly lower missing frequencies. Premium products (300+) present only 20 missing items per 1,000 orders, while items priced between 100–300 show the lowest rate at 6.9.

However, the financial impact follows the opposite pattern. Despite their lower missing frequency, premium products generate the largest revenue loss, totaling 118,803, far exceeding the losses associated with lower-priced items.

Overall, the results indicate that operational risk is concentrated in lower-priced products, while financial risk is primarily driven by high-value items.

H8 Within electronics, watches are more reported as missing item, while, within supermarket, beverages are more reported as missing item.

```
In [175... df_h8_1 = df_products_long[df_products_long['macro_category'] == 'Electronics'].
               'product_name')['macro_category'].count().sort_values(ascending=False).reset
df_h8_2 = df_products_long[df_products_long['macro_category'] == 'Supermarket'].
               'product_name')['macro_category'].count().sort_values(ascending=False).reset
display_side_by_side([df_h8_1, df_h8_2], ['Items Missing by Electronics Products
```

Items Missing by Electronics Products

	product_name	count
0	Bose QuietComfort Earbuds	14
1	Logitech Mouse	14
2	Dell XPS 13	12
3	Beats Studio Pro Headphone	11
4	Nest Thermostat	11
5	Dyson V11 Vacuum	10
6	JBL Flip Speaker	10
7	HP Envy Printer	10
8	Roku Ultra Streaming Device	10
9	Panasonic Lumix Camera	9

Items Missing by Supermarket Products

	product_name	count
0	Chicken Breast	19
1	Ground Coffee	19
2	Cheddar Cheese	18
3	Frozen Shrimp	17
4	Uncured Bacon	17
5	Peanut Butter	17
6	Canned Tuna	17
7	Rice Cakes	16
8	Turkey Burgers	16
9	Broccoli	15

This hypothesis is not supported.

The hypothesis that watches are the most frequently reported missing items within electronics and beverages within supermarket is not supported by the data.

Within the electronics category, the most frequently missing products are consumer electronics accessories and devices, including Bose QuietComfort Earbuds, Logitech

Mouse, Dell XPS 13, and Beats Studio Pro Headphones. Watches do not appear among the most frequently missing items.

Within the supermarket category, the most reported missing products are primarily protein and pantry items such as Chicken Breast, Ground Coffee, Cheddar Cheese, and Frozen Shrimp. Beverages do not appear among the most frequently missing products.

These results suggest that missing item patterns are more strongly associated with specific high-demand or frequently purchased products rather than broader product types such as watches or beverages.

H9: Missing item rates and revenue loss rates differ significantly across customer age groups.

In [176...

```

df_merge_2 = pd.merge(df_merge_ord_miss_prod, df_customers_4, on='customer_id',
df_h9 = df_merge_2
order_h9 = ['Young Adult', 'Adult', 'Middle Age', 'Senior']

df_h9 = hypothesis_metrics(df_h9,'customer_age_group',include_missing=True,inclu
df_h9 = df_h9.sort_values(by='customer_age_group',key=lambda x: pd.Categorical(x

plt.figure(figsize=(18, 6))

# 1. Incidence Rate (Frequency of occurrence)
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h9.set_index('customer_age_group')['incidence_rate'].plot(kind='bar', ax=ax1,
df_h9.set_index('customer_age_group')['revenue_loss_total'].plot(kind='line',ax=
ax1.set_title('Incidence Rate vs Total Revenue Loss by customer_age_group')
ax1.set_ylabel('Incidence Rate')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

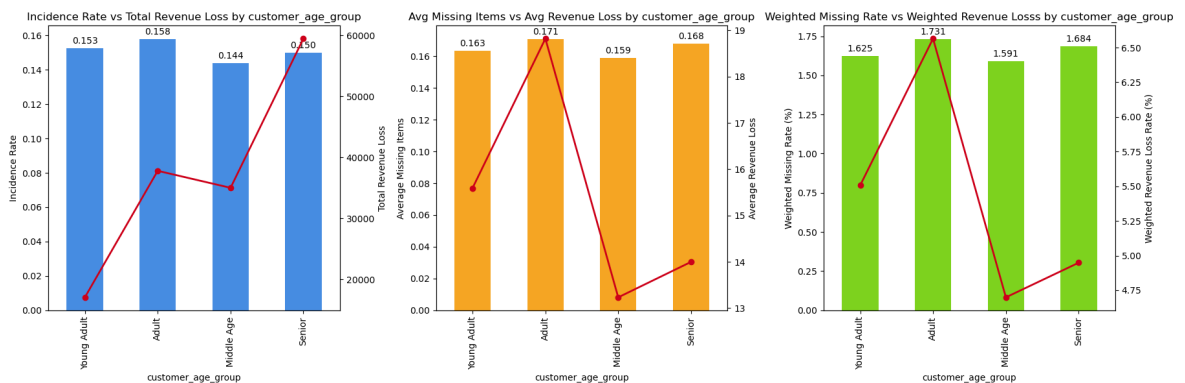
# 2. Average Missing Items (Error intensity)
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h9.set_index('customer_age_group')['avg_missing'].plot(kind='bar',ax=ax2,colo
df_h9.set_index('customer_age_group')['revenue_loss_mean'].plot(kind='line',ax=a
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by customer_age_group')
ax2.set_ylabel('Average Missing Items')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h9.set_index('customer_age_group')['miss_item_weighted_rate_%'].plot(kind='ba
df_h9.set_index('customer_age_group')['rev_loss_weighted_rate_%'].plot(kind='lin
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by customer_age_g
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h9)

```



Out[176...

	customer_age_group	total_orders	incidence_rate	avg_missing	total_missing	std_mi
0	Young Adult	1095	0.1525	0.1635	179	0.40
1	Adult	2009	0.1578	0.1707	343	0.41
2	Middle Age	2647	0.1439	0.1590	421	0.40
3	Senior	4249	0.1499	0.1680	714	0.42

This hypothesis is weakly / partially supported.

The hypothesis that missing item rates and revenue loss rates differ significantly across customer age groups receives limited support from the data.

Missing item rates remain relatively consistent across age groups, ranging from 1.59% among middle-aged customers to 1.73% among adults. These differences are relatively small, suggesting that operational issues are fairly evenly distributed across customer segments.

However, revenue loss rates show greater variation. The adult segment presents the highest revenue loss rate at 6.56%, followed by young adults at 5.51%. Middle-aged and senior customers show lower financial impact levels, with rates of 4.70% and 4.95%, respectively.

Overall, while missing item rates do not vary substantially across age groups, financial impact appears slightly higher among adult customers.

H10: Drivers associated with multiple IDs exhibit higher missing item and revenue loss rates than drivers with a single ID.

In [177...

```

df_merge_3 = pd.merge(df_merge_ord_miss_prod, df_drivers_4, on='driver_id', how='left')
df_h10 = df_merge_3

df_h10 = (df_h10.groupby('driver_id_type').agg(
    total_orders=('order_id', 'count'),
    total_missing=('missing_flag', 'sum'),
    avg_missing=('missing_flag', 'mean'),
    revenue_loss_total=('revenue_loss', 'sum'),
    total_order_amount=('order_amount', 'sum')
))

df_h10['revenue_loss_mean'] = (df_h10['revenue_loss_total'] / df_h10['total_order_amount'])

df_h10 = df_h10.sort_values('avg_missing')

plt.figure(figsize=(18,6))

# 1 Total missing
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h10['avg_missing'].plot(kind='bar', ax=ax1, color='#F5A623')
df_h10['revenue_loss_mean'].plot(kind='line', ax=ax1_1, marker='o', linewidth=2)
ax1.set_title('Avg Missing Items vs Avg Revenue Loss by Driver ID Type')
ax1.set_ylabel('Average of Missing Items')
ax1_1.set_ylabel('Average of Revenue Loss')
bar_value(ax1, decimals=4)

```

```
plt.tight_layout()
plt.show()

df_h10_1 = df_h10.sort_values('avg_missing', ascending=False)
style_hypothesis_table(df_h10_1)
```



Out[177...

	total_orders	total_missing	avg_missing	revenue_loss_total	total_order_a
driver_id_type					
Multiple IDs	76	20	0.2632	1,920.8900	24568.1
Single ID	9924	1482	0.1493	147,461.8000	2808453.1

This hypothesis is supported, but with limited evidence.

Drivers associated with multiple IDs exhibit higher missing item rates and higher revenue loss rates compared to drivers operating under a single ID. The average missing rate for multiple-ID drivers reaches 0.263 missing items per order, compared to 0.149 for single-ID drivers.

Similarly, the revenue loss rate is higher among drivers with multiple IDs, reaching 7.82% compared to 5.25% for drivers with a single ID.

However, this group represents a very small portion of the dataset, with only 76 orders, which limits the statistical reliability of the comparison. While the results suggest a potential operational risk associated with drivers using multiple IDs, further investigation with a larger sample would be necessary to confirm the relationship.

H11: Missing item rates and revenue loss rates differ significantly across driver age groups.

In [178...

```
df_h11 = df_merge_3
order_h13 = ['Young', 'Adult', 'Experienced', 'Senior']

df_h11 = hypothesis_metrics(df_h11, 'driver_age_group', include_missing=True, inclu
df_h11 = df_h11.sort_values(by='driver_age_group', key=lambda x: pd.Categorical(x

plt.figure(figsize=(18, 6))

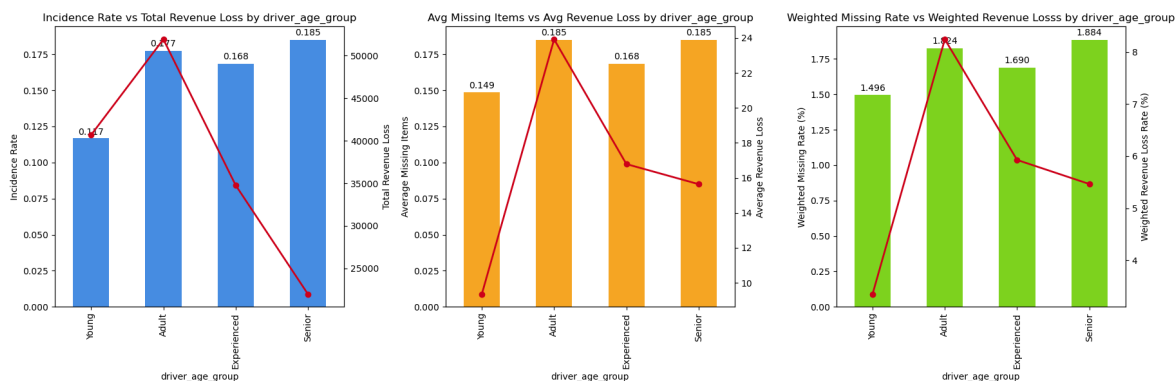
# 1. Incidence Rate (Frequency of occurrence)
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h11.set_index('driver_age_group')['incidence_rate'].plot(kind='bar', ax=ax1, c
df_h11.set_index('driver_age_group')['revenue_loss_total'].plot(kind='line', ax=a
ax1.set_title('Incidence Rate vs Total Revenue Loss by driver_age_group')
ax1.set_ylabel('Incidence Rate')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

# 2. Average Missing Items (Error intensity)
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h11.set_index('driver_age_group')['avg_missing'].plot(kind='bar', ax=ax2, color
df_h11.set_index('driver_age_group')['revenue_loss_mean'].plot(kind='line', ax=ax
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by driver_age_group')
ax2.set_ylabel('Average Missing Items')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h11.set_index('driver_age_group')['miss_item_weighted_rate_%'].plot(kind='bar
df_h11.set_index('driver_age_group')['rev_loss_weighted_rate_%'].plot(kind='line
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by driver_age_gro
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h11)
```



Out[178...

	driver_age_group	total_orders	incidence_rate	avg_missing	total_missing	std_missin
0	Young	4351	0.1168	0.1487	647	0.44276
1	Adult	2171	0.1773	0.1847	401	0.40670
2	Experienced	2072	0.1684	0.1684	349	0.37434
3	Senior	1406	0.1849	0.1849	260	0.38837

This hipoteses is supported.

Missing item rates and revenue loss rates vary across driver age groups. Senior drivers present the highest missing item rate at 1.88%, followed closely by adult drivers at 1.82%. Young drivers show the lowest missing rate at 1.50%.

Revenue loss rates show even stronger variation across groups. Adult drivers present the highest financial impact with a revenue loss rate of 8.24%, while young drivers show the lowest rate at 3.35%.

These results suggest that operational performance and financial impact may vary across driver experience profiles, potentially reflecting differences in workload, route complexity, or operational behavior.

H12: Missing item rate and Revenue Loss rate per trip increases with driver trip volume.

In [179...

```

df_h12 = df_merge_3
order_h14 = ['Low (11-24 trips)', 'Medium (25-41 trips)', 'High (42-60 trips)']

df_h12 = hypothesis_metrics(df_h12, 'trip_bin', include_missing=True, include_reven
df_h12 = df_h12.sort_values(by='trip_bin', key=lambda x: pd.Categorical(x, catego

plt.figure(figsize=(18, 6))

# 1. Incidence Rate (Frequency of occurrence)
ax1 = plt.subplot(1,3,1)
ax1_1 = ax1.twinx()
df_h12.set_index('trip_bin')['incidence_rate'].plot(kind='bar', ax=ax1,color='#4
df_h12.set_index('trip_bin')['revenue_loss_total'].plot(kind='line',ax=ax1_1,mar
ax1.set_title('Incidence Rate vs Total Revenue Loss by trip_bin')
ax1.set_ylabel('Incidence Rate')
ax1_1.set_ylabel('Total Revenue Loss')
bar_value(ax1, decimals=3)

```

```

# 2. Average Missing Items (Error intensity)
ax2 = plt.subplot(1,3,2)
ax2_1 = ax2.twinx()
df_h12.set_index('trip_bin')['avg_missing'].plot(kind='bar',ax=ax2,color='#F5A62')
df_h12.set_index('trip_bin')['revenue_loss_mean'].plot(kind='line',ax=ax2_1,mark
ax2.set_title('Avg Missing Items vs Avg Revenue Loss by trip_bin')
ax2.set_ylabel('Average Missing Items')
ax2_1.set_ylabel('Average Revenue Loss')
bar_value(ax2, decimals=3)

# 3. Weighted Missing Rate (Weighted Rate - Real Impact).
ax3 = plt.subplot(1,3,3)
ax3_1 = ax3.twinx()
df_h12.set_index('trip_bin')['miss_item_weighted_rate_%'].plot(kind='bar',ax=ax3
df_h12.set_index('trip_bin')['rev_loss_weighted_rate_%'].plot(kind='line',ax=ax3
ax3.set_title('Weighted Missing Rate vs Weighted Revenue Losss by trip_bin')
ax3.set_ylabel('Weighted Missing Rate (%)')
ax3_1.set_ylabel('Weighted Revenue Loss Rate (%)')
bar_value(ax3, decimals=3)

plt.tight_layout()
plt.show()

style_hypothesis_table(df_h12)

```



Out[179...

	trip_bin	total_orders	incidence_rate	avg_missing	total_missing	std_missing	total_c
0	Low (11–24 trips)	2477	0.1413	0.1587	393	0.414132	
1	Medium (25–41 trips)	2654	0.1688	0.1865	495	0.435287	
2	High (42–60 trips)	2483	0.1514	0.1663	413	0.412489	
3	Very High (61–78 trips)	2386	0.1375	0.1492	356	0.391141	

This hypothesis is not supported.

The hypothesis that missing item rates and revenue loss rates increase with driver trip volume is not supported by the data.

Missing item rates do not increase consistently with higher trip volumes. The highest missing rate occurs in the medium trip group (1.89%), while drivers with very high trip volumes show one of the lowest rates (1.51%).

Revenue loss rates also show a decreasing trend as trip volume increases. Drivers with low trip volumes present the highest revenue loss rate at 5.83%, while drivers with very high trip volumes show the lowest rate at 4.53%.

These results suggest that higher trip volume does not lead to increased operational risk. Instead, more active drivers may demonstrate greater operational efficiency or experience, resulting in lower missing item and revenue loss rates.

8.2.3 Additional Analysis (Operational Monitoring)

Top 15 Customers with highest number of items missing reported and revenue loss generated.

```
In [180... df_h9_1 = (df_merge_2.groupby(['customer_id', 'customer_name', 'customer_age']).  
            missing_by_customers=('items_missing', 'sum'),  
            revenue_loss_by_customers=('revenue_loss', 'sum')).reset_index().sort_values  
df_h9_2 = (df_merge_2.groupby(['customer_id', 'customer_name', 'customer_age']).  
            missing_by_customers=('items_missing', 'sum'),  
            revenue_loss_by_customers=('revenue_loss', 'sum')).reset_index().sort_values  
  
display_side_by_side([df_h9_1, df_h9_2], ['Items Missing by Customers', 'Revenue
```


Items Missing by Customers

	customer_id	customer_name	customer_age	missing_by_customers	revenue_loss_by
820	WCID5820	Edward Henry	73	9	
685	WCID5685	Shelley Gould	85	7	
857	WCID5857	Douglas Roth	81	7	
165	WCID5165	David Mccoy	59	6	
593	WCID5593	Victor Gonzalez	86	6	
7	WCID5007	Timothy Daniel	18	6	
339	WCID5339	Jerry Mullen	64	6	
1164	WCID6164	Karen Moore	47	6	
978	WCID5978	Raymond Chavez	30	6	
1096	WCID6096	Holly Stewart	76	5	
46	WCID5046	James Hopkins	38	5	
817	WCID5817	Angela Bradley	67	5	
652	WCID5652	Carla Knox	23	5	
594	WCID5594	Robert Odonnell	45	5	
511	WCID5511	David Hicks	89	5	



Top 15 Drivers with highest number of items missing reported and revenue loss generated.

```
In [181... df_h11_1 = (df_merge_3.groupby(['driver_id', 'driver_name', 'driver_age', 'drive
missing_by_drivers=('items_missing','sum'),
revenue_loss_by_driver=('revenue_loss','sum'))).reset_index().sort_values(by
df_h11_2 = (df_merge_3.groupby(['driver_id', 'driver_name', 'driver_age', 'drive
missing_by_drivers=('items_missing','sum'),
revenue_loss_by_driver=('revenue_loss','sum'))).reset_index().sort_values(by
display_side_by_side([df_h11_1, df_h11_2], ['Items Missing by Drivers', 'Revenue
```

Items Missing by Drivers

	driver_id	driver_name	driver_age	driver_trips	missing_by_drivers	revenue_loss_by
0	WDID09882	Angela Young	18	75	6	
1	WDID09883	Charles Rios DVM	18	62	6	
2	WDID09884	James Winters	18	22	6	
3	WDID09877	James White	18	14	6	
4	WDID09878	Roger Green	18	22	6	
5	WDID09879	Shane Palmer	18	35	6	
6	WDID09880	Brittany Gordon	18	36	6	
7	WDID09876	Jordan Daniel	18	53	6	
8	WDID09875	Stephen Randolph	18	64	6	
9	WDID09874	Billy Lawson	18	37	6	
10	WDID09873	Pamela Moore	18	64	6	
11	WDID09881	Tracey Watkins	18	26	6	
12	WDID09891	Justin Parsons	18	16	5	
13	WDID09890	Cynthia Owens	18	67	5	
14	WDID09893	Mrs. Valerie Schwartz MD	18	77	5	

8.2.4 Integrated Analytical Narrative (H1–H12)

This analysis aimed to understand the drivers behind missing items in e-commerce deliveries by testing twelve hypotheses across operational, product, customer, and driver-related dimensions. The objective was to determine whether missing items are primarily driven by operational factors, behavioral patterns, or potential fraud indicators.

Operational Patterns

The first set of hypotheses examined whether operational conditions influence missing item occurrences.

Contrary to expectations, missing item rates were not higher during night deliveries (H1). Instead, the morning period shows the highest incidence rate, average missing items, and revenue loss rate, suggesting greater operational variability during this time.

Seasonality analysis (H2) also reveals no consistent increase during peak demand periods such as summer or year-end. Missing item rates fluctuate moderately across months, with higher values appearing in specific months rather than during traditional seasonal peaks.

Similarly, the hypothesis that weekends present higher operational risk was not supported (H3). Missing item rates are slightly higher at the beginning of the week, particularly on Mondays, while weekends show mixed results.

Regional analysis (H4) shows moderate but relatively small differences across locations, indicating that missing items are not strongly concentrated in specific regions. Financial impact per incident varies slightly across regions (H5), with Orlando presenting higher average revenue losses, suggesting the presence of higher-value products in those orders.

Overall, operational conditions such as delivery timing or location show moderate influence, but no strong structural patterns.

Product-Level Risk

Product characteristics reveal some of the most significant patterns in the analysis.

Supermarket products account for the majority of missing item incidents (over 80%), reflecting the high operational volume and handling complexity of grocery orders (H6). In contrast, electronics represent a smaller share of missing items but generate substantially higher financial losses due to their higher unit value.

Price-level analysis (H7) reinforces this pattern. Lower-priced products show the highest missing frequencies, while high-value products generate the largest financial impact despite occurring less frequently.

Product-level analysis (H8) also indicates that missing items tend to involve frequently purchased everyday products, rather than specific product types such as watches or beverages.

Together, these findings highlight an important distinction between operational volume risk and financial exposure risk.

Customer Behavior

Customer characteristics show limited influence on missing item patterns.

Missing item rates remain relatively consistent across customer age groups (H9), suggesting that operational issues affect customers broadly rather than being concentrated in specific demographic segments.

However, revenue loss rates are slightly higher among adult customers, which may reflect larger baskets or higher-value purchases within this group.

Overall, customer behavior does not appear to be a primary driver of missing item incidents.

Driver Behavior and Operational Execution

Driver-related factors present more noticeable variation.

Drivers associated with multiple IDs show higher missing item and revenue loss rates (H10), although the limited sample size requires cautious interpretation.

Driver age groups also present differences (H11). Senior and adult drivers show slightly higher missing item rates, while young drivers present the lowest levels. These variations may reflect differences in experience, workload, or operational practices.

Driver workload analysis (H12) reveals that higher trip volume does not increase operational risk. In fact, drivers with very high trip volumes present lower missing item and revenue loss rates, suggesting that experience and routine may improve operational performance.

Overall Interpretation

Across all hypotheses, the results suggest that missing items are primarily driven by operational factors rather than systemic fraud patterns.

Most incidents are associated with high-volume product categories and routine operational processes, while financial exposure is largely driven by high-value items such as electronics.

These findings highlight the importance of distinguishing between operational risk (high-frequency, low-value incidents) and financial risk (low-frequency, high-value events) when designing monitoring and mitigation strategies.

Missing items are primarily operational → No strong evidence of systematic fraud patterns

Incidents driven by high-volume operations → Concentrated in frequent, routine processes

Financial impact driven by high-value products → Low frequency, but high exposure (e.g., electronics)

8.2.5 Hypothesis Key Business Findings

Key Finding 1 — Operational Volume Drives Most Missing Incidents (Lower-priced products drive higher operational error volume)

The majority of missing item incidents are concentrated in low-price, high-frequency supermarket products. These items represent the largest share of operational errors due

to their high transaction volume and frequent handling during order preparation.

Business Implication:

Operational controls should prioritize high-volume product categories, improving picking accuracy, packaging procedures, and order verification processes for frequently purchased items. Even small improvements in these processes could significantly reduce the total number of missing item incidents.

Key Finding 2 — High-Value Electronics Drive Disproportionate Financial Risk (Electronics generate the highest financial risk per incident)

Although electronics represent a smaller share of missing item incidents, they generate significantly higher revenue losses due to their substantially higher unit value.

Business Implication:

High-value items should receive additional operational safeguards, such as enhanced handling procedures, verification checkpoints, or packaging confirmation protocols to reduce financial exposure from high-value missing items.

Key Finding 3 — Financial Impact Varies More Than Incident Frequency Across Regions and Customer Segments

Differences in missing item rates across regions and customer age groups are relatively moderate. This suggests that missing item incidents are not strongly concentrated in specific locations or customer segments.

However, financial impact varies significantly. Certain regions and customer groups generate higher revenue loss per incident. Regional variation highlights concentration of high-value losses. Customer segments differ in financial exposure, not frequency.

Business Implication:

Monitoring should prioritize financial exposure (value per incident) rather than just incident frequency, especially across regions and customer segments.

Key Finding 4 — Driver Experience May Improve Operational Performance

Drivers with higher trip volumes tend to present lower missing item and revenue loss rates, suggesting that operational experience may improve delivery accuracy and efficiency.

Business Implication:

Encouraging driver retention, experience accumulation, and operational training may help reduce missing item incidents over time. Experienced drivers may develop more efficient routines and better familiarity with delivery processes.

Key Finding 5 — Certain Operational Time Windows Show Higher Risk Variability (Operational risk is higher at the start of the week)

Morning deliveries and the beginning of the week show slightly higher missing item rates and financial impact, suggesting that operational variability may be greater during these periods.

This may reflect factors such as higher operational workload, shift transitions, or inventory replenishment cycles.

Business Implication:

Operations teams may benefit from additional monitoring or quality checks during high-variability periods, particularly during early-week operations or morning delivery windows.

Key Finding 6 — Potential Operational Risks in Driver Identity Management

Drivers associated with multiple IDs present higher missing item and revenue loss rates, although the sample size remains small.

While this evidence is limited, the pattern may indicate potential operational irregularities or identity management issues within the driver system.

Business Implication:

Companies may benefit from strengthening driver identity verification and monitoring systems to ensure consistent driver identification and reduce potential operational vulnerabilities.

Overall, the analysis suggests that missing items are primarily driven by operational dynamics related to high-volume product handling and delivery processes, while financial exposure is largely associated with high-value products such as electronics.

8.2.6 Financial Impact

Total Revenue Loss: Missing items generated \$149k in revenue loss across 10k orders, representing ~5.3% of total order value.

9.0 Fraud Risk Score Framework

Final Fraud Risk Score Table, Radar Plot and Heatmap Plot

This table show the most risk attributes by the main metrics. Additional a radar and heatmap radar for each attribute is shown for better visualization.

Main Metrics:

group_col (ex: period) - dimension being analyzed

total_orders - reliability of sample

incidence_rate - operational error frequency

miss_item_weighted_rate_% - operational severity

revenue_loss_mean - financial impact

rev_loss_weighted_rate_% - financial efficiency loss

risk_score_0_100 - final score

risk_level - interpretation

```
In [182... # Fraud Risk Score Framework by unique attributes
# df_risk_period = fraud_risk_framework(df_merge_ord_miss_prod, 'period')
# df_risk_period.head().T

# --/--

# Fraud Risk Score Framework by unique attributes with most important attributes
# cols_report = [
#     'period',
#     'total_orders',
#     'incidence_rate',
#     'miss_item_weighted_rate_%',
#     'revenue_loss_mean',
#     'rev_loss_weighted_rate_%',
#     'risk_score_0_100',
#     'risk_level'
# ]

# df_risk_period_1 = df_risk_period[cols_report]
# df_risk_period_1

# --/--

# Radar and Heatmap plot of the Fraud Risk Score Framework by unique attributes
# plot_radar(df_risk_period, 'period')
# plot_heatmap(df_risk_period, 'period')

# --/--

# Most important datasets and attributes for the fraud risk score framework
datasets = {
    'operational': df_merge_ord_miss_prod,
    'driver': df_merge_3,
    'customer': df_merge_2,
    'product': df_products_long
}

attributes = [
    'period',
    'month',
    'day_of_week',
    'region',
    'macro_category',
    'price_bin',
    'customer_age_group',
    'driver_id_type',
    'driver_age_group',
```

```

'trip_bin']

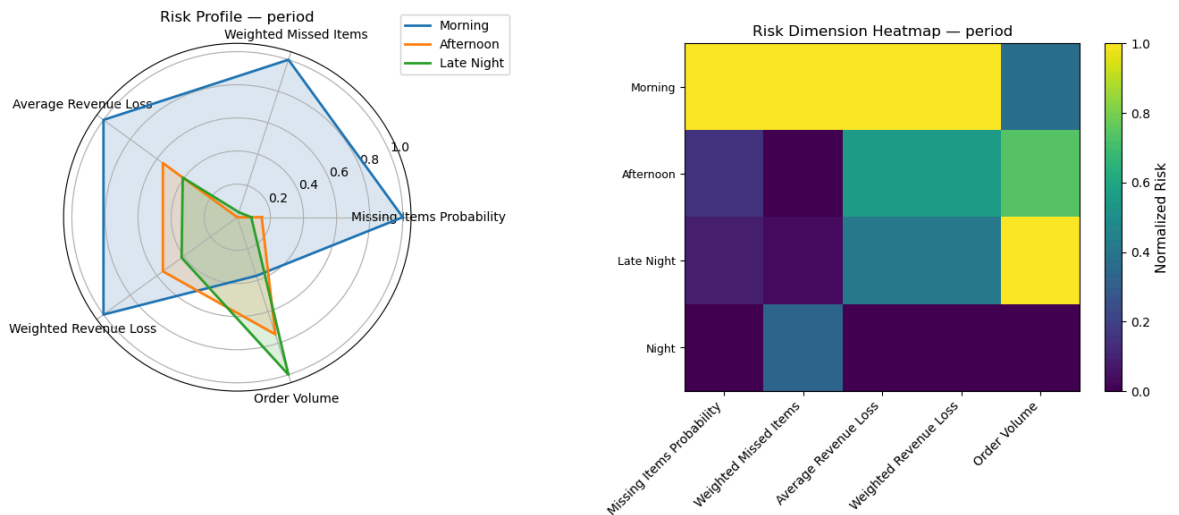
# 1 Risk analysis + plots
df_risk_reports = generate_risk_reports(datasets,attributes)

# 2 Final consolidated table
df_final_risk_summary = create_final_risk_table(df_risk_reports)

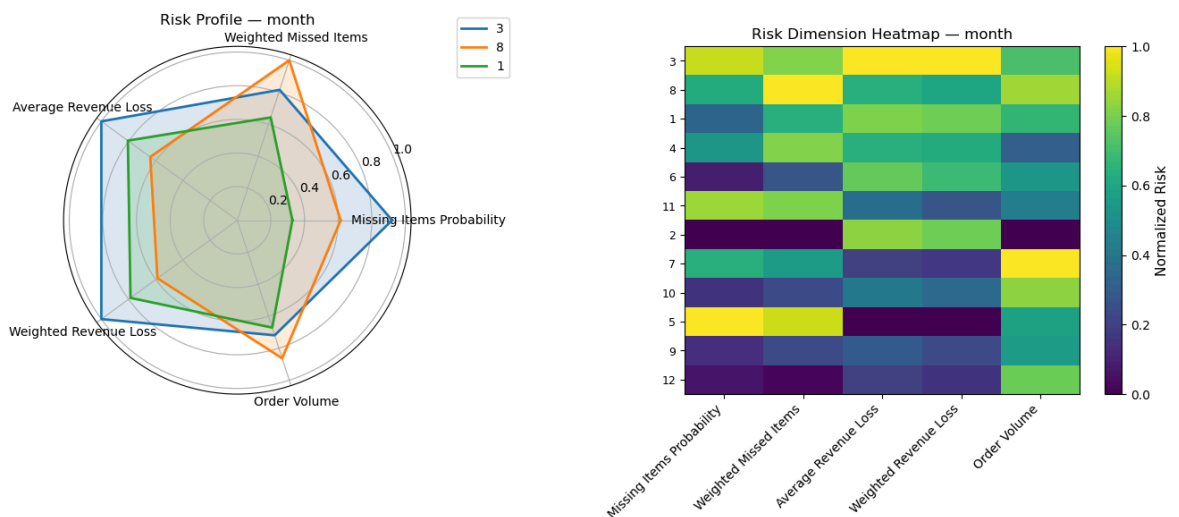
# 3 View most critical segments
df_final_risk_summary.reset_index(drop=True)

```

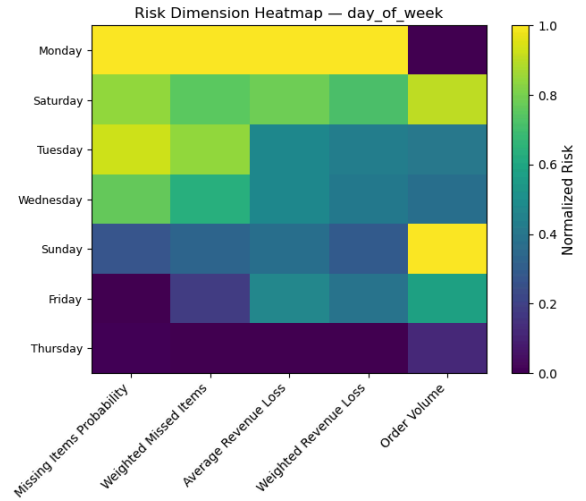
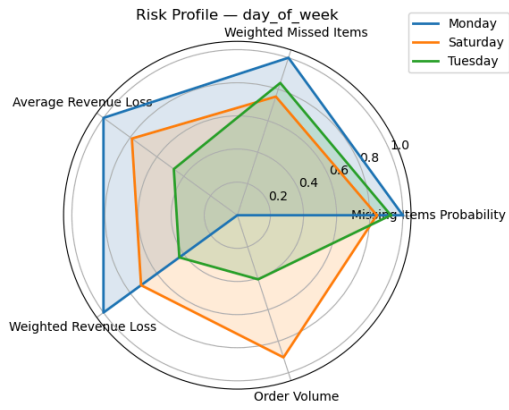
Fraud Risk Analysis → period



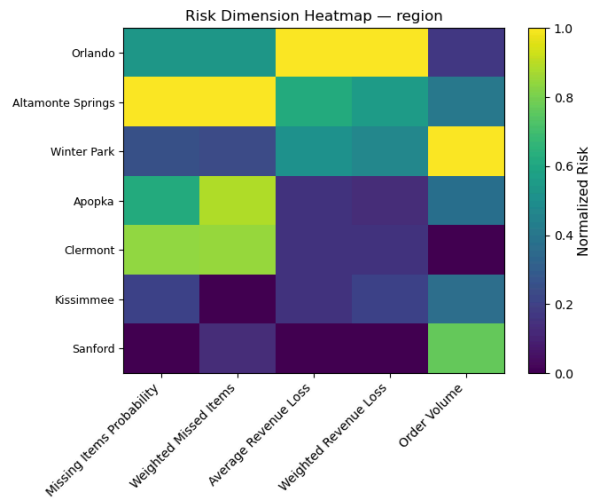
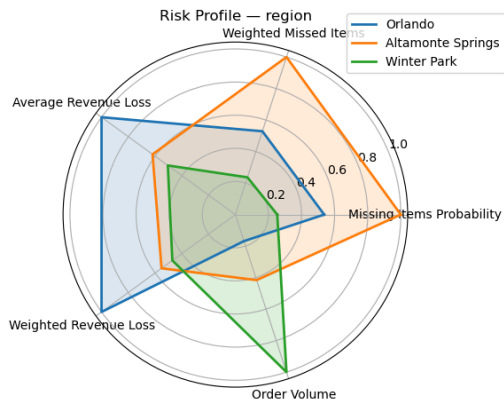
Fraud Risk Analysis → month



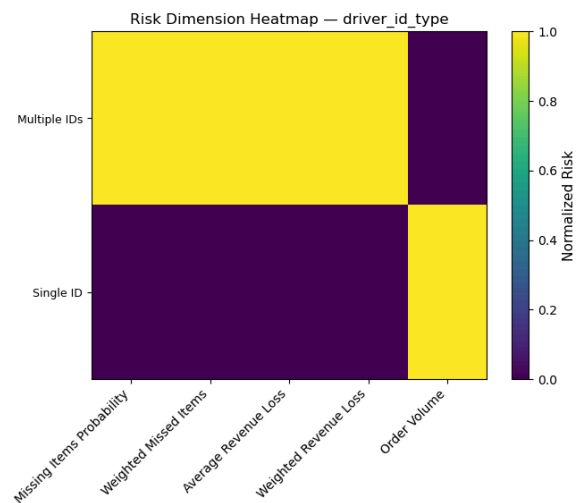
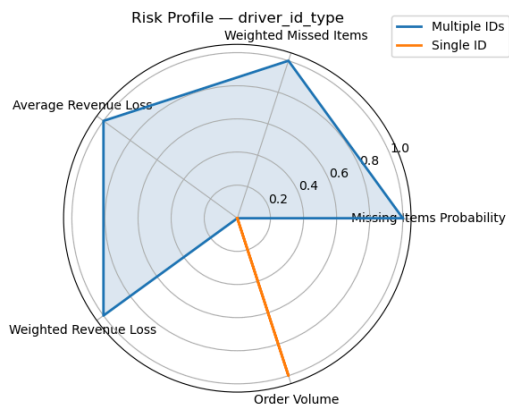
Fraud Risk Analysis → day_of_week



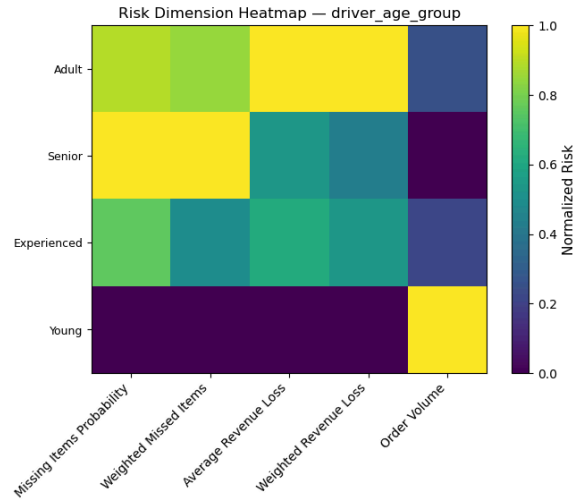
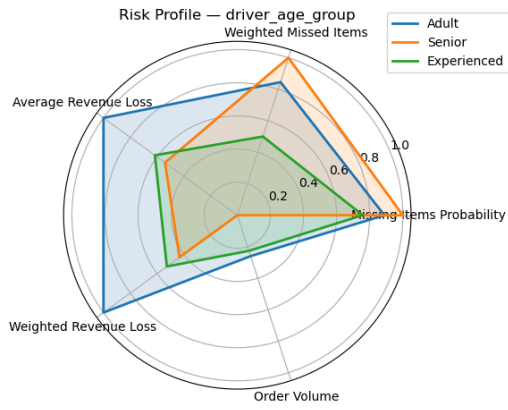
Fraud Risk Analysis → region



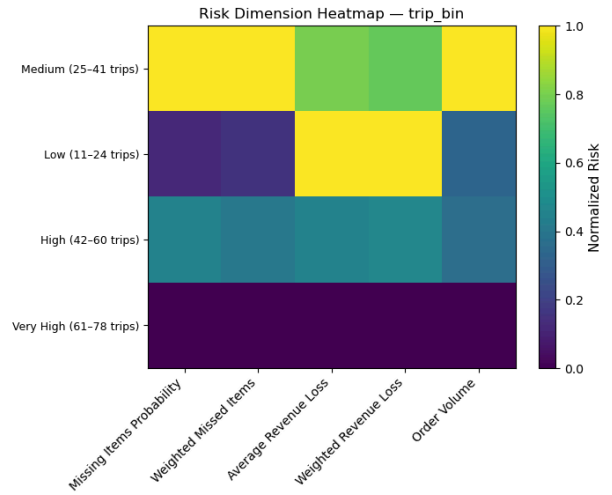
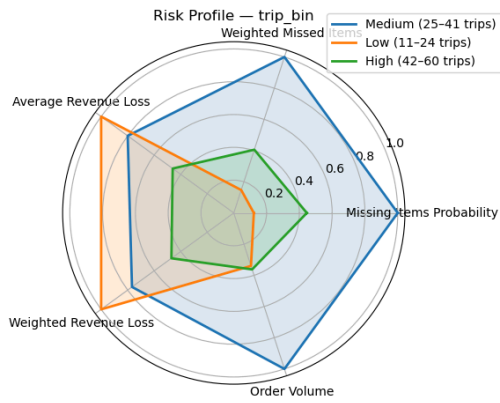
Fraud Risk Analysis → driver_id_type



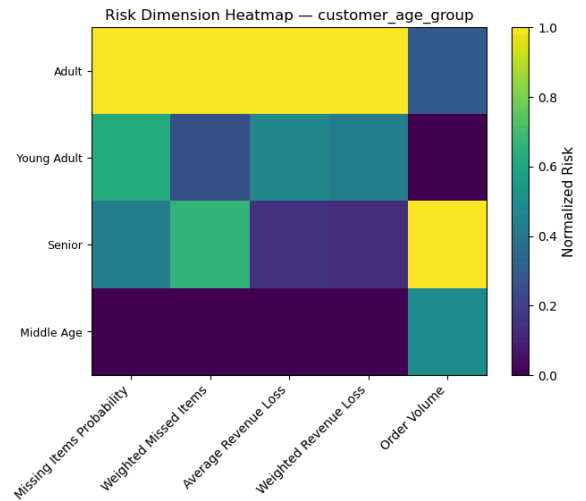
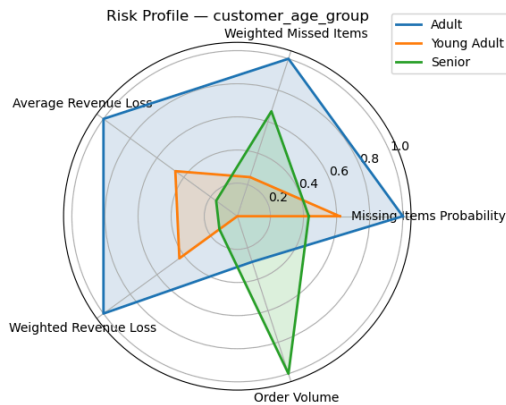
Fraud Risk Analysis → driver_age_group



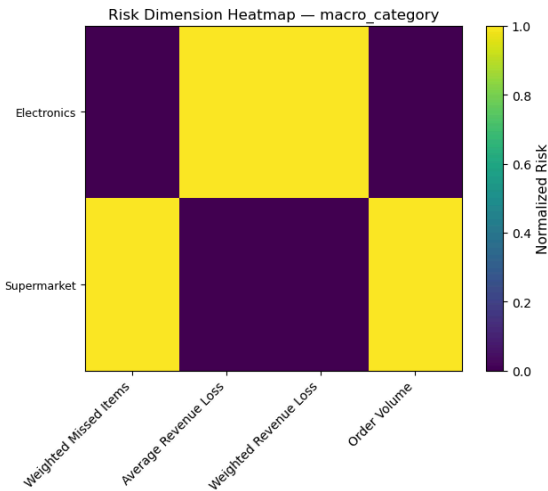
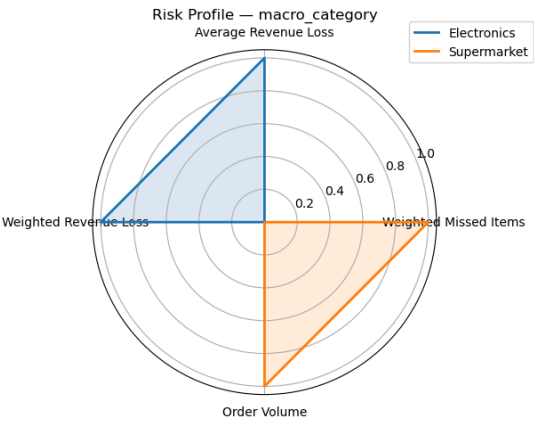
Fraud Risk Analysis → trip_bin



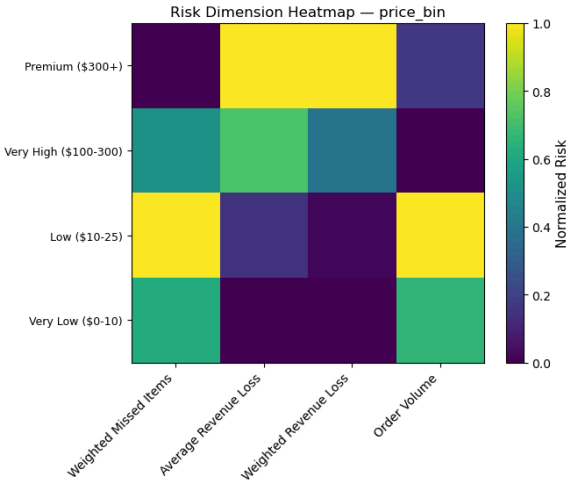
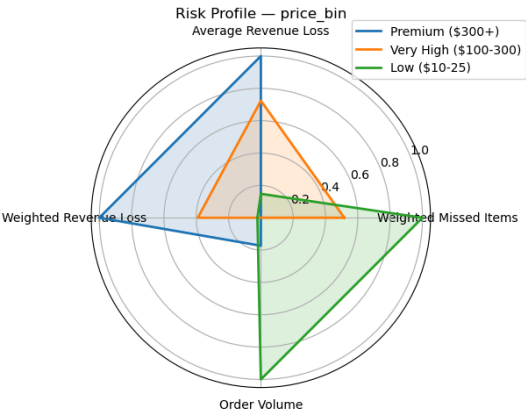
Fraud Risk Analysis → customer_age_group



Fraud Risk Analysis → macro_category



Fraud Risk Analysis → price_bin



Out[182...

	dataset	attribute	segment	total_orders	incidence_rate	miss_item
0	operational	period	Morning	2481	0.161225	
1	operational	month	3	850	0.167059	
2	customer	customer_age_group	Adult	2009	0.157790	
3	driver	driver_id_type	Multiple IDs	76	0.263158	
4	operational	day_of_week	Monday	1374	0.160844	
5	driver	driver_age_group	Adult	2171	0.177338	
6	driver	trip_bin	Medium (25–41 trips)	2654	0.168802	
7	operational	day_of_week	Saturday	1476	0.157182	
8	operational	region	Orlando	1401	0.151320	
9	operational	month	8	871	0.156142	
10	operational	region	Altamonte Springs	1426	0.161992	
11	operational	month	1	843	0.145907	
12	driver	trip_bin	Low (11–24 trips)	2477	0.141300	
13	product	price_bin	Premium (\$300+)	200	1.000000	
14	operational	month	4	789	0.153359	
15	product	macro_category	Electronics	269	1.000000	
16	driver	driver_age_group	Senior	1406	0.184922	
17	operational	day_of_week	Tuesday	1420	0.159155	
18	driver	driver_age_group	Experienced	2072	0.168436	
19	operational	month	6	823	0.137303	
20	operational	day_of_week	Wednesday	1416	0.155367	
21	product	price_bin	Very High (\$100-300)	69	1.000000	
22	operational	month	11	808	0.164604	

	dataset	attribute	segment	total_orders	incidence_rate	miss_item
23	operational	month	2	745	0.134228	
24	operational	region	Winter Park	1485	0.144781	
25	product	price_bin	Low (\$10-25)	826	1.000000	
26	driver	trip_bin	High (42-60 trips)	2483	0.151430	
27	operational	period	Afternoon	2526	0.147664	
28	customer	customer_age_group	Young Adult	1095	0.152511	
29	product	macro_category	Supermarket	1394	1.000000	
30	operational	day_of_week	Sunday	1487	0.143914	
31	operational	month	7	891	0.157127	
32	operational	month	10	866	0.139723	
33	operational	period	Late Night	2557	0.146656	
34	customer	customer_age_group	Senior	4249	0.149918	
35	operational	region	Apopka	1422	0.153305	
36	operational	day_of_week	Friday	1440	0.137500	
37	operational	month	5	830	0.169880	
38	operational	region	Clermont	1384	0.158237	
39	operational	month	9	826	0.139225	
40	product	price_bin	Very Low (\$0-10)	568	1.000000	
41	operational	month	12	858	0.136364	
42	operational	region	Kissimmee	1421	0.143561	
43	driver	driver_id_type	Single ID	9924	0.149335	
44	driver	driver_age_group	Young	4351	0.116755	
45	operational	region	Sanford	1461	0.138946	
46	customer	customer_age_group	Middle Age	2647	0.143937	
47	operational	period	Night	2436	0.145320	
48	operational	day_of_week	Thursday	1387	0.137707	

	dataset	attribute	segment	total_orders	incidence_rate	miss_item
49	driver	trip_bin	Very High (61–78 trips)	2386	0.137469	

Top 5 Fraud Risk Groups

```
In [183... df_top_groups = generate_fraud_groups(df_final_risk_summary, top_n=5).reset_index()
df_top_groups
```

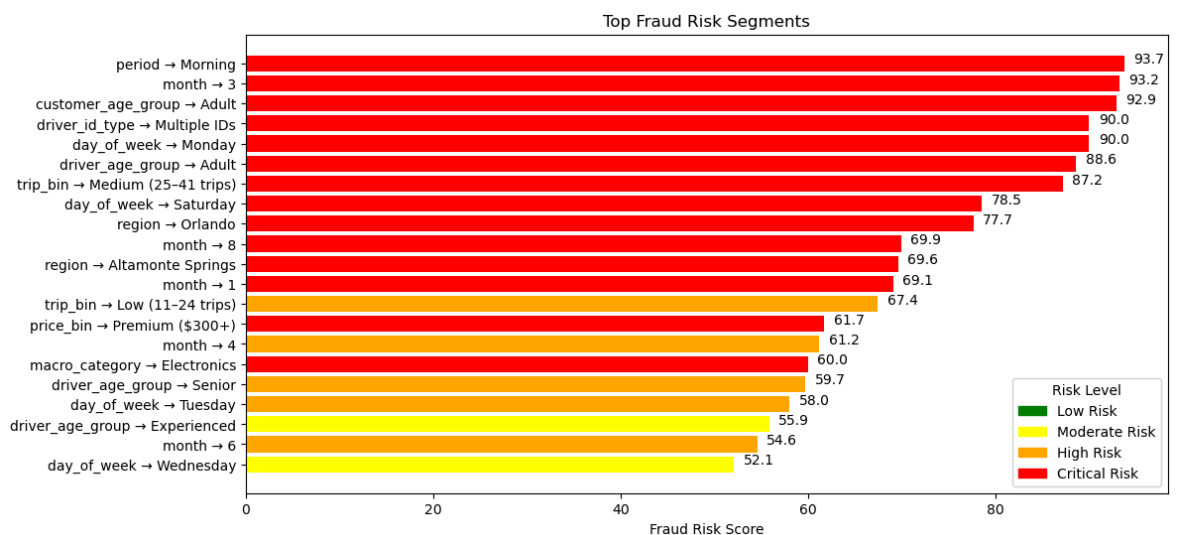
	fraud_driver	risk_score_0_100	risk_level
0	period → Morning	93.719007	Critical Risk
1	month → 3	93.230365	Critical Risk
2	customer_age_group → Adult	92.897906	Critical Risk
3	driver_id_type → Multiple IDs	90.000000	Critical Risk
4	day_of_week → Monday	89.999999	Critical Risk

Top Fraud Risk Groups

- Morning orders show the highest fraud risk concentration
- Adult customers present elevated missing-item severity and significant revenue loss
- Medium-trip drivers show higher operational anomalies
- March exhibits the highest revenue loss intensity
- Drivers using multiple IDs show abnormal patterns

Fraud Risk Leaderboard

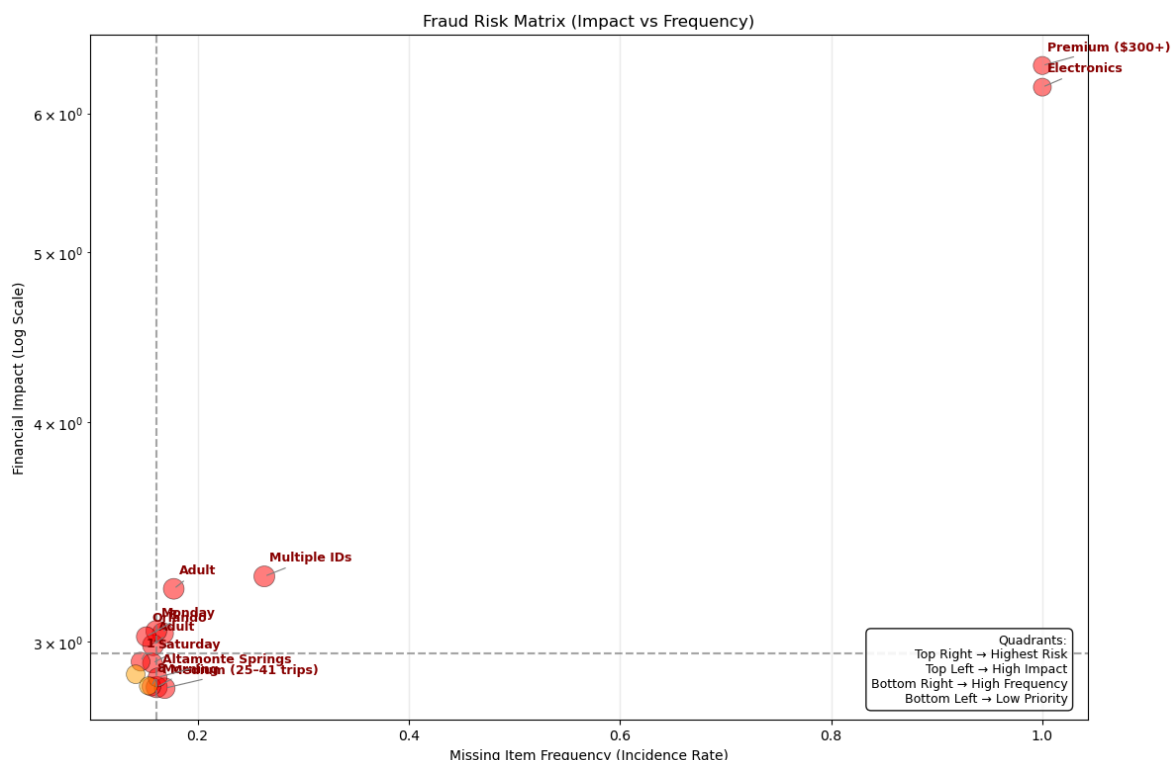
```
In [184... plot_risk_leaderboard(df_final_risk_summary, top_n=21)
```



Fraud Risk Matrix (Impact × Frequency)

In [185...

```
plot_fraud_risk_matrix(df_final_risk_summary, top_n=16)
```



9.2 Fraud Risk Score Key Business Findings

1 - Products and Price Bins at the Top of Risk

- macro_category = Electronics and price_bin = Premium (\$300+) rank as Critical Risk.
- Indicates that high-value tech products have the largest financial impact when missing.
- Insight: Missing high-priced items should be closely monitored.

2 - Specific Operational Periods Are Critical

- period = Morning shows Critical Risk with 2481 affected orders and ~\$39k revenue loss.
- month = 3 and month = 8 appear among high-risk segments → indicates seasonality in missing items.
- day_of_week = Monday and Saturday also show high risk → specific days may have operational weaknesses.

3 - Drivers and Customers Contribute to Risk

- driver_age_group = Adult and Senior, trip_bin = Medium (25–41 trips) show High Risk, but slightly lower than premium products.

- customer_age_group = Adult and Senior also appear in elevated risk → customer type and behavior influence missing item frequency, though financial impact is lower.

4 - Volume vs. Risk

- Premium products have lower volume (200 for Premium, 269 for Electronics), yet are Critical Risk due to high item value.
- High-volume segments like Adult customers or Afternoon orders have moderate/critical risk mostly driven by missing frequency, not monetary value.

Key Insight: High volume does not always equal higher financial risk; for high-value products, even a few missing items significantly raise risk.

5 - Clear Segmentation for Action

- Three main fraud risk drivers identified:
 1. High-value / tech products → financial risk
 2. Specific periods and dates → operational risk
 3. Driver or customer profile → operational/recurrence risk

Perfect insight for the "Top Fraud Drivers" slide, showing what impacts the most and where to focus prevention.

10.0 Final Operational Recommendations

Recommendation 1 — Protect High-Value Electronics

Rationale: H6 & H7. Fraud Risk Insights show that Electronics and Premium items have the largest financial losses, even with lower volume.

Actions:

- Enforce delivery confirmation (photo + verification)
 - Photo of the recipient holding the package
 - PIN sent via SMS to the buyer 30 minutes before arrival
 - Recipient ID check
- Use tamper-proof packaging
 - Seals applied across the package

Recommendation 2 — Monitor Drivers with Multiple IDs

Rationale: H10 and Risk Framework show that drivers with multiple IDs are associated with higher missing item frequency and revenue loss.

- Flag multiple ID accounts
- Automatic flagging and manual audit if 2+ driver names share the same Device ID (IMEI) or Bank Account.
- Implement identity verification
- Random 3D Selfies to prevent account sharing and "Ghost Drivers."

Recommendation 3 — Enhance Operational Accuracy for High-Volume / Risky Products

Rationale: H6 & H7 + Fraud Risk Insights indicate high-volume supermarket items are frequently missing, though individually low-value.

Actions:

- Add picking validation
- Mandatory barcode validation at every picking stage to eliminate item errors.
- User automated order checks
- Automated systems to cross-verify order contents against shipping manifests.

Recommendation 4 — Targeted Driver Performance Management

Rationale: H11 show that driver experience (adult and older) affect operational performance; Risk Framework highlights specific risk segments by driver age and trip bins.

Actions:

- Train mid-experience drivers
- Focused coaching for established drivers to reduce delivery errors.
- Incentivize delivery accuracy
- Tiered bonuses for high verification compliance and positive customer ratings.

Recommendation 5 — Implement Fraud Risk Monitoring Dashboard

Rationale: H1, H2, H3, H6, H7, H9, H11 and Fraud Risk Framework allows real-time identification of high-risk orders, highlighting critical products, periods, and driver/customer segments.

Action:

- Track high-risk segments in real time

→ Instant notifications when orders enter designated "Red Flag" sectors.

- Monitor products, drivers, and time windows

→ Auditing the overlap of high-value goods, new drivers, and morning/early week shift windows.

Recommendation - Hypotheses

1 - H6, H7, H4, H5

2 - H10

3 - H6, H7, H8

4 - H11, H12

5 - H1, H2, H3, H4, H5, H6, H7, H8, H9, H11

11.0 Fraud Risk Score Simulator

A fraud risk simulator was developed using the Fraud Risk Framework scores. The tool estimates the risk level of a delivery based on operational, driver, customer, and product characteristics by aggregating segment-level risk scores.

Link: <https://delivery-fraud-risk-score-simulator-walmart.streamlit.app/>

In [186...

```
# Creating the Values by quartile
q1 = df_final_risk_summary["risk_score_0_100"].quantile(0.25)
q2 = df_final_risk_summary["risk_score_0_100"].quantile(0.50)
q3 = df_final_risk_summary["risk_score_0_100"].quantile(0.75)

def risk_level(score, q1, q2, q3):

    if score <= q1:
        return "Low Risk"

    elif score <= q2:
        return "Moderate Risk"

    elif score <= q3:
        return "High Risk"

    else:
        return "Critical Risk"

# Creating the simulator
def simulate_risk(inputs, risk_table):

    scores = []

    for attr, value in inputs.items():

        row = risk_table[
            (risk_table["attribute"] == attr) &
```

```

        (risk_table["segment"] == value)
    ]

    if not row.empty:
        scores.append(row["risk_score_0_100"].values[0])

    final_score = sum(scores) / len(scores)

    return final_score

# Selecting the attributes
inputs = {
    "period": "Afternoon",
    "month": "2",
    "day_of_week": "Monday",
    "region": "Altamonte Springs ",
    "macro_category": "Electronincs",
    "price_bin": "Premium ($300+)",
    "customer_age_group": "Adult",
    "driver_id_type": "Multiple IDs",
    "driver_age_group": "Experienced",
    "trip_bin": "Very High (61-78 trips) "
}

score = simulate_risk(inputs, df_final_risk_summary)

# Classifying the risk score
score

```

Out[186... np.float64(72.25062260159)

11.1 Online Frau Risk Score Simulator

```

In [187... # Exporting the the risk fraud dataset
# df_final_risk_summary.to_csv('df_final_risk_summary.csv', index=False)

# Creating the app online using Streamlit
import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import os

# -----
# PAGE CONFIG
# -----

st.set_page_config(
    page_title="Delivery Fraud Risk Simulator",
    layout="wide"
)

# -----
# LOAD DATA
# -----

data_path = 'C:/Users/Igor/Repos/Walmart-Delivery-Fraud-Detection/fraud_risk_pro
df_risk = pd.read_csv(data_path + "df_final_risk_summary.csv")

# base_path = os.path.dirname(__file__)

```

```

# data_path = os.path.join(base_path, "data", "df_final_risk_summary.csv")
# df_risk = pd.read_csv(data_path)

# quartile thresholds (same logic as fraud framework)
q1 = df_risk["risk_score_0_100"].quantile(0.25)
q2 = df_risk["risk_score_0_100"].quantile(0.50)
q3 = df_risk["risk_score_0_100"].quantile(0.75)

# -----
# FUNCTIONS
# -----

def risk_level(score):

    if score <= q1:
        return "Low Risk"

    elif score <= q2:
        return "Moderate Risk"

    elif score <= q3:
        return "High Risk"

    else:
        return "Critical Risk"

def get_segment_score(attribute, segment):

    row = df_risk[
        (df_risk["attribute"] == attribute) &
        (df_risk["segment"] == segment)
    ]

    if len(row) > 0:
        return row["risk_score_0_100"].values[0]

    return None

# -----
# RISK LEVEL RECOMENDATIONS
# -----

def risk_recommendation(level):

    if level == "Critical Risk":
        return """
🚨 **Critical Risk Actions**
- Enforce delivery confirmation (photo + PIN +Verification)
- Use tamper-proof packaging for high-value items
- Flag multiple drivers ID accounts
- Track high-risk segments in real time
        """

    elif level == "High Risk":
        return """
⚠️ **High Risk Actions**
- Add picking validation
- User automated order checks
        """

```

```

        elif level == "Moderate Risk":
            return """
        ⚙️ **Moderate Risk Actions**
        - Standard monitoring
        - Track for pattern escalation
            """

        else:
            return """
        ✅ **Low Risk**
        - No immediate action required
            """

# -----
# TITLE
# -----

st.title("🚚 Delivery Fraud Risk Simulator")

st.write(
    """
    This simulator estimates delivery fraud risk using the **Fraud Risk Framework**.
    Select operational, customer, driver, and product characteristics to evaluate th
    """
)

# -----
# INPUTS
# -----

col1, col2, col3 = st.columns(3)

with col1:

    period = st.selectbox(
        "Delivery Period",
        df_risk[df_risk["attribute"]=="period"]["segment"].unique()
    )

    month = st.selectbox(
        "Month",
        df_risk[df_risk["attribute"]=="month"]["segment"].unique()
    )

    day_of_week = st.selectbox(
        "Day of Week",
        df_risk[df_risk["attribute"]=="day_of_week"]["segment"].unique()
    )

with col2:

    region = st.selectbox(
        "Region",
        df_risk[df_risk["attribute"]=="region"]["segment"].unique()
    )

    customer_age = st.selectbox(
        "Customer Age Group",
        df_risk[df_risk["attribute"]=="customer_age_group"]["segment"].unique()

```

```

    )

    driver_id = st.selectbox(
        "Driver ID Type",
        df_risk[df_risk["attribute"]=="driver_id_type"]["segment"].unique()
    )

with col3:

    driver_age = st.selectbox(
        "Driver Age Group",
        df_risk[df_risk["attribute"]=="driver_age_group"]["segment"].unique()
    )

    trip_bin = st.selectbox(
        "Driver Trip Bin",
        df_risk[df_risk["attribute"]=="trip_bin"]["segment"].unique()
    )

    price_bin = st.selectbox(
        "Product Price Bin",
        df_risk[df_risk["attribute"]=="price_bin"]["segment"].unique()
    )

    macro_category = st.selectbox(
        "Product Category",
        df_risk[df_risk["attribute"]=="macro_category"]["segment"].unique()
    )

# -----
# CALCULATE RISK
# -----

if st.button("Calculate Fraud Risk"):

    attributes = {

        "period":period,
        "month":month,
        "day_of_week":day_of_week,
        "region":region,

        "customer_age_group":customer_age,

        "driver_id_type":driver_id,
        "driver_age_group":driver_age,
        "trip_bin":trip_bin,

        "price_bin":price_bin,
        "macro_category":macro_category
    }

    results = []

    for attr, seg in attributes.items():

        score = get_segment_score(attr, seg)

        if score is not None:

```

```

        results.append({
            "attribute":attr,
            "segment":seg,
            "risk_score":score
        })

df_results = pd.DataFrame(results)

final_score = df_results["risk_score"].mean()

level = risk_level(final_score)

# -----
# RESULTS
# -----

st.subheader("Fraud Risk Score")

col1, col2 = st.columns(2)

with col1:

    st.metric("Risk Score", round(final_score,1))

    if level == "Critical Risk":
        st.error(level)

    elif level == "High Risk":
        st.warning(level)

    elif level == "Moderate Risk":
        st.info(level)

    else:
        st.success(level)

# -----
# GAUGE CHART
# -----

with col2:

    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=final_score,
        title={'text':"Fraud Risk Score"},
        gauge={
            'axis':{'range':[0,100]},
            'bar':{'color':"darkred"},
            'steps':[
                {'range':[0,25], 'color':"green"},
                {'range':[25,50], 'color':"yellow"},
                {'range':[50,75], 'color':"orange"},
                {'range':[75,100], 'color':"red"}
            ]
        }
    ))

    st.plotly_chart(fig, use_container_width=True)

```

```
# -----  
# RECOMMENDED ACTIONS  
# -----  
  
    st.subheader("Recommended Actions")  
    st.info(risk_recommendation(level))  
  
# -----  
# RISK BREAKDOWN  
# -----  
  
    st.subheader("Risk Contribution by Factor")  
  
    st.dataframe(df_results)  
  
# -----  
# TOP RISK DRIVERS  
# -----  
  
    st.subheader("Top Risk Drivers")  
  
    top_risk = df_results.sort_values(  
        "risk_score",  
        ascending=False  
    ).head(5)  
  
    st.dataframe(top_risk)
```


[illegible]

[illegible]

```
2026-03-27 17:57:15.953 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.954 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.956 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.957 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.958 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.959 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.962 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.964 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.964 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.966 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.968 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.970 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.972 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.974 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.978 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.980 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.982 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.983 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.984 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.987 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.988 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.990 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.992 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.995 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.996 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.997 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
2026-03-27 17:57:15.998 Thread 'MainThread': missing ScriptRunContext! This warni
ng can be ignored when running in bare mode.
```

A fraud risk simulator was developed using the Fraud Risk Framework scores. The tool estimates the risk level of a delivery based on operational, driver, customer, and product characteristics by aggregating segment-level risk scores.

Link: <https://delivery-fraud-risk-score-simulator-walmart.streamlit.app/>